

01	FILE	FILE 01 main.js 启动入口与云环境初始化
02	1	// 【性能优化】生产环境静默日志输出
03	2	import { silenceConsoleInProduction } from '@/utils/logger.js';
04	3	silenceConsoleInProduction();
05	4	
06	5	// 字体预加载 - 确保汇文明朝字体在应用启动时就加载
07	6	// #ifdef H5
08	7	if (typeof document !== 'undefined') {
09	8	console.log('📄 [字体预加载] H5端开始预加载汇文明朝字体');
10	9	
11	10	// 创建字体预加载链接
12	11	const fontLink = document.createElement('link');
13	12	fontLink.rel = 'preload';
14	13	fontLink.as = 'font';
15	14	fontLink.type = 'font/otf';
16	15	fontLink.href = '/static/fonts/Huiwen-mincho.otf';
17	16	fontLink.crossOrigin = 'anonymous';
18	17	document.head.appendChild(fontLink);
19	18	
20	19	// 同时创建字体样式表
21	20	const fontStyle = document.createElement('style');
22	21	fontStyle.textContent = `
23	22	@font-face {
24	23	font-family: '汇文明朝';
25	24	src: url('/static/fonts/Huiwen-mincho.otf') format('opentype');
26	25	font-weight: normal;
27	26	font-style: normal;
28	27	font-display: swap;
29	28	}
30	29	`;
31	30	document.head.appendChild(fontStyle);
32	31	
33	32	// 使用 FontFace API 强制加载字体
34	33	if (typeof FontFace !== 'undefined') {
35	34	const font = new FontFace('汇文明朝', 'url(/static/fonts/Huiwen-mincho.otf)');
36	35	font.load().then(function(loader) {
37	36	document.fonts.add(loader);
38	37	console.log('✅ [字体预加载] H5端汇文明朝字体加载成功');
39	38	}).catch(function(error) {
40	39	console.error('❌ [字体预加载] H5端汇文明朝字体加载失败:', error);
41	40	});
42	41	}
43	42	}
44	43	// #endif
45	44	
46	45	// #ifdef APP-PLUS
47	46	// App端字体预加载
48	47	console.log('📄 [字体预加载] App端开始预加载汇文明朝字体');
49	48	try {

涉及文件: main.js

回车键Poementer软件

源码范围: main.js:50 -> main.js:98

01	50	console.log('🔴 [字体预加载] App端字体路径:', fontPath);
02	51	
03	52	// 使用 uni.loadFontFace 加载字体
04	53	uni.loadFontFace({
05	54	family: '汇文明朝',
06	55	source: `url("\${fontPath}")`,
07	56	global: true,
08	57	success: function() {
09	58	console.log('✅ [字体预加载] App端汇文明朝字体加载成功');
10	59	},
11	60	fail: function(err) {
12	61	console.error('❌ [字体预加载] App端汇文明朝字体加载失败:', err);
13	62	}
14	63	});
15	64	} catch (e) {
16	65	console.warn('❌ [字体预加载] App端字体预加载失败:', e);
17	66	}
18	67	// #endif
19	68	
20	69	// #ifdef MP-WEIXIN
21	70	// 微信小程序端不预加载字体, 使用系统默认字体
22	71	console.log('⚠️ [字体预加载] 小程序端跳过字体预加载, 使用系统字体');
23	72	// #endif
24	73	
25	74	import App from './App';
26	75	import fileUrlCache from './_utils/file-url-cache';
27	76	import { setupCacheEventBridges } from '@api-cache/events.js';
28	77	import { getMyInfo } from '@api-cache/my.js';
29	78	
30	79	// 全局mixins, 用于实现setData等功能, 请勿删除! ';
31	80	import zpMixins from '@uni_modules/zp-mixins/index.js';
32	81	
33	82	// #ifndef VUE3
34	83	import Vue from 'vue';
35	84	
36	85	// --- TCB 初始化开始 (这是我们新加的部分) ---
37	86	// 1. 引入新的、正确的 tencent-cloudbase SDK
38	87	// #ifdef H5 APP-PLUS
39	88	import tcb from '@cloudbase/js-sdk';
40	89	// #endif
41	90	
42	91	// 2. 初始化云开发环境 (支持热更新/重复执行, 确保单例)
43	92	// #ifdef H5 APP-PLUS
44	93	const __global = (typeof globalThis !== 'undefined') ? globalThis : (typeof window !== 'undefined' ? window : {});
45	94	const tcbApp = __global.__tcbAppInstance tcb.init({
47	95	env: 'cloud1-5gb0pbyl400845f5',
48	96	auth: { persistence: 'local' },
49	97	timeout: 120000

涉及文件: main.js

回车键Poementer软件

源码范围: main.js:99 -> main.js:148

01	99	if (!__global.__tcbAppInstance) {
02	100	__global.__tcbAppInstance = tcbApp;
03	101	}
04	102	// #endif
05	103	
06	104	// 3. 将 tcb 实例挂载到 Vue 的原型上
07	105	// #ifdef H5 APP-PLUS
08	106	Vue.prototype.\$tcb = tcbApp;
09	107	// #endif
10	108	
11	109	// 小程序环境: 初始化 wx.cloud 并挂载到 Vue 原型
12	110	// 【关键修复】不使用条件编译, 改为运行时检测
13	111	if (typeof wx !== 'undefined' && wx.cloud) {
14	112	console.log('🟡 [main.js] 检测到微信小程序环境, 开始初始化云开发...');
15	113	try {
16	114	// 立即同步初始化 wx.cloud
17	115	wx.cloud.init({
18	116	env: 'cloud1-5gb0pbyl400845f5',
19	117	traceUser: true
20	118	});
21	119	console.log('✅ [main.js] 云开发初始化完成, 环境 ID: cloud1-5gb0pbyl400845f5');
22	120	
23	121	// 挂载 wx.cloud 到 Vue 原型, 保持与 H5/APP 的一致性
24	122	Vue.prototype.\$tcb = {
25	123	callFunction(options = {}) {
26	124	console.log('🟡 [Vue.\$tcb] 调用云函数:', options.name);
27	125	return wx.cloud.callFunction(options);
28	126	},
29	127	getTempFileURL(args = {}) { return wx.cloud.getTempFileURL(args); },
30	128	database() { return wx.cloud.database(); },
31	129	uploadFile(options = {}) { return wx.cloud.uploadFile(options); },
32	130	downloadFile(options = {}) { return wx.cloud.downloadFile(options); },
33	131	deleteFile(options = {}) { return wx.cloud.deleteFile(options); },
34	132	// 添加 auth 方法的空实现, 避免调用时报错
35	133	auth() {
36	134	return {
37	135	currentUser: null,
38	136	signInAnonymously() {
39	137	return Promise.resolve();
40	138	}
41	139	};
42	140	}
43	141	};
44	142	console.log('✅ [main.js] wx.cloud 已挂载到 Vue.prototype.\$tcb');
45	143	
46	144	// 同时挂载到 uni 对象, 确保全局可访问
47	145	if (typeof uni !== 'undefined') {
48	146	uni.\$tcb = Vue.prototype.\$tcb;
49	147	console.log('✅ [main.js] wx.cloud 已挂载到 uni.\$tcb');

涉及文件: main.js

回车键Poementer软件

源码范围: main.js:149 -> main.js:197

01	149	} catch (error) {
02	150	console.error('✖ [main.js] 云开发初始化失败:', error);
03	151	}
04	152	} else if (typeof wx !== 'undefined') {
05	153	console.error('✖ [main.js] wx.cloud 不可用, 请使用 2.2.3 或以上的基础库');
06	154	}
07	155	
08	156	// 注入临时 URL 解析器 (全平台支持)
09	157	try {
10	158	fileUrlCache.setResolver(async (ids) => {
11	159	if (!Array.isArray(ids) ids.length === 0) return {};
12	160	
13	161	let res;
14	162	// #ifdef H5 APP-PLUS
15	163	if (typeof tcbApp !== 'undefined' && tcbApp.getTempFileURL) {
16	164	res = await tcbApp.getTempFileURL({ fileList: ids });
17	165	}
18	166	// #endif
19	167	
20	168	// #ifdef MP-WEIXIN
21	169	if (typeof wx !== 'undefined' && wx.cloud && wx.cloud.getTempFileURL) {
22	170	res = await wx.cloud.getTempFileURL({ fileList: ids });
23	171	}
24	172	// #endif
25	173	
26	174	const map = {};
27	175	if (res && Array.isArray(res.fileList)) {
28	176	res.fileList.forEach((it) => {
29	177	if (it && it.tempFileURL) {
30	178	map[it.fileID] = { url: it.tempFileURL, maxAgeSec: it.maxAge 3600 };
31	179	}
32	180	});
33	181	}
34	182	return map;
35	183	});
36	184	} catch (e) {
37	185	console.warn('fileUrlCache resolver setup failed:', e);
38	186	}
39	187	
40	188	
41	189	// 5. 立即进行匿名认证, 确保在调用云函数之前完成认证 (幂等处理)
42	190	// 这是关键修复: 在应用启动时就完成 TCB 匿名认证, 避免后续调用云函数时出现 "you can't request wi
43	191	thout auth" 错误
44	191	// #ifdef H5 APP-PLUS
45	192	(async () => {
46	193	try {
47	194	// 检查是否已经登录
48	195	const currentUser = tcbApp.auth().currentUser;
49	196	if (!currentUser) {

涉及文件: main.js

回车键Poementer软件

源码范围: main.js:198 -> main.js:245

01	198	}
02	199	} catch (error) {
03	200	console.error('✖ [TCB初始化] 匿名认证失败:', error);
04	201	// 认证失败不应该阻止应用启动, 只是会影响后续的云函数调用
05	202	}
06	203	})();
07	204	// #endif
08	205	Vue.prototype.\$requireOpenid = function () {
09	206	const appInstance = getApp();
10	207	let openid = appInstance && appInstance.globalData && appInstance.globalData.openid;
11	208	// H5/App 刷新后先从本地缓存兜底恢复
12	209	if (!openid) {
13	210	try {
14	211	openid = uni.getStorageSync('userOpenId') uni.getStorageSync('openid');
15	212	if (openid) {
16	213	appInstance.globalData = appInstance.globalData {};
17	214	appInstance.globalData.openid = openid;
18	215	}
19	216	} catch (e) {}
20	217	}
21	218	if (!openid) {
22	219	// 检查是否是应用启动初期 (避免在自动登录过程中显示提示)
23	220	const isAppStarting = !appInstance.globalData !appInstance.globalData._loginProcessComple
24	221	ted;
25	221	if (!isAppStarting) {
26	222	uni.showToast({ title: '用户未登录', icon: 'none' });
27	223	}
28	224	}
29	225	return openid;
30	226	};
31	227	// --- TCB 初始化结束 ---
32	228	
33	229	// 缓存事件桥: 监听发帖/头像更新等事件并失效相关缓存
34	230	try { setupCacheEventBridges(); } catch (e) { console.warn('setupCacheEventBridges failed', e);
35	231	}
36	231	
37	232	// 延迟初始化未读消息红点管理器 (优化启动速度)
38	233	// 在页面加载完成后再检查未读消息, 不阻塞启动流程
39	234	setTimeout(() => {
40	235	try {
41	236	const unreadBadge = require('@/cache/stores/unread-badge.js');
42	237	unreadBadge.initUnreadCount();
43	238	} catch (e) { console.warn('unreadBadge init failed', e); }
44	239	try {
45	240	const activityBadge = require('@/cache/stores/activity-badge.js');
46	241	activityBadge.initActivityBadge();
47	242	} catch (e) { console.warn('activityBadge init failed', e); }
48	243	}, 1000); // 延迟1秒, 让页面先加载完成
49	244	

涉及文件: main.js

源码范围: main.js:246 -> main.js:290

01	246	try {
02	247	const waitLoginThen = (fn) => {
03	248	const start = Date.now();
04	249	const MAX = 5000; // 最多等待 5s
05	250	const timer = setInterval(() => {
06	251	try {
07	252	const app = getApp() && getApp();
08	253	const done = app && app.globalData && app.globalData._loginProcessCompleted;
09	254	const oid = app && app.globalData && app.globalData.openid;
10	255	if ((done && oid) Date.now() - start > MAX) {
11	256	clearInterval(timer);
12	257	fn && fn();
13	258	}
14	259	} catch (_) { clearInterval(timer); }
15	260	}, 120);
16	261	};
17	262	
18	263	waitLoginThen(async () => {
19	264	try {
20	265	const ctx = { \$tcb: (Vue && Vue.prototype && Vue.prototype.\$tcb) ? Vue.prototype.\$tcb : (t
21	266	ypeof uni !== 'undefined' && uni.\$tcb ? uni.\$tcb : null) };
22	266	// 触发一次资料读取（内部会将 cloud:// 头像映射为 https，并写入持久层）
23	267	await getMyInfo(ctx);
24	268	} catch (e) {
25	269	console.warn('🚩 [prewarm] getMyInfo failed', e);
26	270	}
27	271	});
28	272	} catch (e) { console.warn('prewarm setup failed', e); }
29	273	
30	274	// 调试: 读取本地 CACHE_DEBUG 开关并注入查看函数
31	275	try {
32	276	const cacheManager = require('@/_utils/cache-manager');
33	277	let dbg = false;
34	278	try { const v = uni.getStorageSync() && uni.getStorageSync('CACHE_DEBUG'); dbg = v === true v
35	279	=== '1' v === 1 v === 'true'; } catch(_) {}
36	279	cacheManager.setDebug(!dbg);
37	280	if (typeof uni !== 'undefined') {
38	281	uni.\$cacheStats = () => { try { console.log('[CacheStats]', cacheManager.getStats()); } catc
39	282	h(e) { console.log('[CacheStats] failed', e); } };
40	282	uni.\$cacheDebug = (on) => { try { cacheManager.setDebug(!on); console.log('[CacheDebug] =',
41	283	!!on); } catch(e) {} };
42	283	}
43	284	} catch(e) { console.warn('cache debug setup failed', e); }
44	285	
45	286	function resolveOpenidForCall(functionName) {
46	287	// 登录/获取 openid 类函数允许在无 openid 时调用
47	288	// getPhoneNumberByToken 是 uniCloud 云函数，仅用于获取手机号，不需要腾讯云开发的 openid
48	289	if (functionName === 'login' functionName === 'getOpenId' functionName === 'getPhoneNumb
49	290	erByToken') {

涉及文件: main.js

回车键Poementer软件

源码范围: main.js:291 -> main.js:339

01	291	}
02	292	const appInstance = getApp();
03	293	let openid = appInstance.globalData && appInstance.globalData.openid;
04	294	// 刷新后优先从本地恢复 openid, 并回填到 globalData, 保证早期请求可用
05	295	if (!openid) {
06	296	try {
07	297	openid = uni.getStorageSync('userOpenId') uni.getStorageSync('openid');
08	298	if (openid) {
09	299	appInstance.globalData = appInstance.globalData {};
10	300	appInstance.globalData.openid = openid;
11	301	}
12	302	} catch (e) {}
13	303	}
14	304	if (!openid) {
15	305	// 检查是否是应用启动初期（避免在自动登录过程中显示提示）
16	306	const isAppStarting = !appInstance.globalData !appInstance.globalData._loginProcessComple
17	307	ted;
18	307	if (!isAppStarting) {
19	308	uni.showToast({ title: '用户未登录', icon: 'none' });
20	309	}
21	310	return { openid: null, allowed: false };
22	311	}
23	312	return { openid, allowed: true };
24	313	}
25	314	
26	315	// #ifdef APP-PLUS
27	316	// uniCloud 拦截器仅在 APP 环境下启用（用于热更新和一键登录）
28	317	const originalUniCloudCallFunction = uniCloud.callFunction.bind(uniCloud);
29	318	uniCloud.callFunction = function (options = {}) {
30	319	const name = options.name;
31	320	if (!name) {
32	321	return originalUniCloudCallFunction(options);
33	322	}
34	323	const { openid, allowed } = resolveOpenidForCall(name);
35	324	if (!allowed) {
36	325	const error = new Error('NO_OPENID');
37	326	error.code = 'NO_OPENID';
38	327	if (typeof options.fail === 'function') {
39	328	options.fail(error);
40	329	}
41	330	if (typeof options.complete === 'function') {
42	331	options.complete(error);
43	332	}
44	333	return Promise.reject(error);
45	334	}
46	335	const data = Object.assign({}, options.data {});
47	336	if (openid) {
48	337	data.openid = openid;
49	338	}

涉及文件: main.js

回车键Poementer软件

源码范围: main.js:340 -> main.js:389

01	340	return originalUniCloudCallFunction(mergedOptions);
02	341	};
03	342	// #endif
04	343	
05	344	// #ifdef H5 APP-PLUS
06	345	const originalTcbCallFunction = tcbApp.callFunction.bind(tcbApp);
07	346	tcbApp.callFunction = function (options = {}) {
08	347	const name = options.name;
09	348	if (!name) {
10	349	return originalTcbCallFunction(options);
11	350	}
12	351	const { openid, allowed } = resolveOpenidForCall(name);
13	352	if (!allowed) {
14	353	const error = new Error('NO_OPENID');
15	354	error.code = 'NO_OPENID';
16	355	if (typeof options.fail === 'function') {
17	356	options.fail(error);
18	357	}
19	358	if (typeof options.complete === 'function') {
20	359	options.complete(error);
21	360	}
22	361	return Promise.reject(error);
23	362	}
24	363	const data = Object.assign({}, options.data {});
25	364	if (openid) {
26	365	data.openid = openid;
27	366	}
28	367	const mergedOptions = Object.assign({}, options, { data });
29	368	return originalTcbCallFunction(mergedOptions);
30	369	};
31	370	// #endif
32	371	
33	372	Vue.use(zpMixins); // 保留这行
34	373	
35	374	Vue.config.productionTip = false;
36	375	App.mpType = 'app';
37	376	const app = new Vue({
38	377	...App
39	378	});
40	379	app.\$mount();
41	380	// #endif
42	381	
43	382	// #ifdef VUE3
44	383	import { createSSRApp } from 'vue';
45	384	// VUE3环境下的TCB引入
46	385	// #ifdef H5 APP-PLUS
47	386	import tcb from '@cloudbase/js-sdk';
48	387	// #endif
49	388	

涉及文件: main.js

回车键Poementer软件

源码范围: main.js:390 -> main.js:439

01	390	const app = createSSRApp(App);
02	391	
03	392	// --- VUE3 环境下的 TCB 初始化 (为您一并写好, 以备未来升级) ---
04	393	// #ifdef H5 APP-PLUS
05	394	// 1. 初始化
06	395	const tcbApp = tcb.init({
07	396	env: 'cloud1-5gb0pbyl400845f5', // 使用正确的环境ID
08	397	// 启用匿名认证, 允许未登录用户调用云函数
09	398	auth: {
10	399	persistence: 'local'
11	400	},
12	401	timeout: 120000
13	402	});
14	403	// 2. 挂载
15	404	app.config.globalProperties.\$tcb = tcbApp;
16	405	
17	406	// 3. 立即进行匿名认证, 确保在调用云函数之前完成认证
18	407	(async () => {
19	408	try {
20	409	const currentUser = tcbApp.auth().currentUser;
21	410	if (!currentUser) {
22	411	await tcbApp.auth().signInAnonymously();
23	412	}
24	413	} catch (error) {
25	414	console.error('✖ [TCB初始化-VUE3] 匿名认证失败:', error);
26	415	}
27	416	})();
28	417	// #endif
29	418	// #ifdef MP-WEIXIN
30	419	// 小程序环境: 初始化 wx.cloud 并挂载到 app
31	420	if (!wx.cloud) {
32	421	console.error('✖ [云开发-VUE3] 请使用 2.2.3 或以上的基础库以使用云能力');
33	422	} else {
34	423	console.log('🌱 [云开发-VUE3] 开始初始化...');
35	424	wx.cloud.init({
36	425	env: 'cloud1-5gb0pbyl400845f5', // 您的云环境 ID
37	426	traceUser: true
38	427	});
39	428	console.log('✅ [云开发-VUE3] 初始化完成, 环境 ID: cloud1-5gb0pbyl400845f5');
40	429	}
41	430	app.config.globalProperties.\$tcb = {
42	431	callFunction(options = {}) { return wx.cloud.callFunction(options); },
43	432	getTempFileURL(args = {}) { return wx.cloud.getTempFileURL(args); },
44	433	database() { return wx.cloud.database(); },
45	434	uploadFile(options = {}) { return wx.cloud.uploadFile(options); },
46	435	downloadFile(options = {}) { return wx.cloud.downloadFile(options); },
47	436	deleteFile(options = {}) { return wx.cloud.deleteFile(options); },
48	437	// 添加 auth 方法的空实现, 避免调用时报错
49	438	auth() {

涉及文件: main.js

回车键Poementer软件

源码范围: main.js:440 -> main.js:489

01	440	currentUser: null,
02	441	signInAnonymously() {
03	442	return Promise.resolve();
04	443	}
05	444	};
06	445	}
07	446	};
08	447	
09	448	// 注入临时 URL 解析器 (VUE3环境)
10	449	try {
11	450	fileUrlCache.setResolver(async (ids) => {
12	451	if (!Array.isArray(ids) ids.length === 0) return {};
13	452	let res;
14	453	// #ifdef H5 APP-PLUS
15	454	if (typeof tcbApp !== 'undefined' && tcbApp.getTempFileURL) {
16	455	res = await tcbApp.getTempFileURL({ fileList: ids });
17	456	}
18	457	// #endif
19	458	// #ifdef MP-WEIXIN
20	459	if (typeof wx !== 'undefined' && wx.cloud && wx.cloud.getTempFileURL) {
21	460	res = await wx.cloud.getTempFileURL({ fileList: ids });
22	461	}
23	462	// #endif
24	463	const map = {};
25	464	if (res && Array.isArray(res.fileList)) {
26	465	res.fileList.forEach((it) => {
27	466	if (it && it.tempFileURL) {
28	467	map[it.fileID] = { url: it.tempFileURL, maxAgeSec: it.maxAge 3600 };
29	468	}
30	469	});
31	470	}
32	471	return map;
33	472	});
34	473	} catch (e) {
35	474	console.warn('fileUrlCache resolver setup failed (VUE3):', e);
36	475	}
37	476	// #endif
38	477	app.config.globalProperties.\$requireOpenid = function () {
39	478	const appInstance = getApp();
40	479	const openid = appInstance && appInstance.globalData && appInstance.globalData.openid;
41	480	if (!openid) {
42	481	uni.showToast({ title: '用户未登录', icon: 'none' });
43	482	}
44	483	return openid;
45	484	};
46	485	// --- TCB 初始化结束 ---
47	486	
48	487	app.mixin(zpMixins); // 保留这行
49	488	return {

涉及文件: main.js -> App.vue (共 2 个文件)

回车键Poementer软件

源码范围: main.js:490 -> App.vue:46

01	490	};
02	491	}
03	492	// #endif
04	FILE	FILE 02 App.vue 应用级生命周期与全局逻辑
05	1	<script>
06	2	// #ifdef APP-PLUS
07	3	// import checkUpdate from '@uni_modules/uni-upgrade-center-app/utls/check-update';
08	4	import { checkAndUpdate } from '@utls/hotUpdate.js';
09	5	// #endif
10	6	import fontManager from '@utls/fontManager.js';
11	7	
12	8	export default {
13	9	// 【重构】1. 将所有全局数据放入 data 函数中, 这是 Vue 的标准做法
14	10	data() {
15	11	return {
16	12	// globalData 对象将在这里被 Vue 响应式地管理
17	13	globalData: {
18	14	userInfo: null,
19	15	openid: null
20	16	// 您可以保留其他全局变量, 但 env ID 已在 main.js 中配置, 这里不再需要
21	17	}
22	18	};
23	19	},
24	20	
25	21	// 【重构】2. onLaunch 是 Vue 的生命周期函数, 保持不变
26	22	onLaunch: function (options) {
27	23	// 小程序环境: 在 App.vue 中初始化云开发 (确保最早执行)
28	24	// 不使用条件编译, 改为运行时检测
29	25	if (typeof wx !== 'undefined' && wx.cloud) {
30	26	console.log('🟡 [App.vue] 检测到微信小程序环境, 开始初始化云开发...');
31	27	try {
32	28	// 立即初始化 wx.cloud
33	29	wx.cloud.init({
34	30	env: 'cloud1-5gb0pbyl400845f5',
35	31	traceUser: true
36	32	});
37	33	console.log('✅ [App.vue] 云开发初始化完成');
38	34	
39	35	// 挂载到 this.\$tcb
40	36	this.\$tcb = wx.cloud;
41	37	console.log('✅ [App.vue] wx.cloud 已挂载到 this.\$tcb');
42	38	
43	39	// 同时挂载到 Vue 原型和 uni 对象
44	40	if (typeof Vue !== 'undefined' && Vue.prototype) {
45	41	Vue.prototype.\$tcb = wx.cloud;
46	42	console.log('✅ [App.vue] wx.cloud 已挂载到 Vue.prototype.\$tcb');
47	43	}
48	44	if (typeof uni !== 'undefined') {
49	45	uni.\$tcb = wx.cloud;

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:47 -> App.vue:95

01	47	}
02	48	} catch (error) {
03	49	console.error('✖ [App.vue] 云开发初始化失败:', error);
04	50	}
05	51	} else if (typeof wx !== 'undefined') {
06	52	console.error('✖ [App.vue] wx.cloud 不可用, 请检查基础库版本 (需要 >= 2.2.3)');
07	53	}
08	54	
09	55	// #ifdef APP-PLUS
10	56	// 处理 URL Scheme 启动 (GitHub OAuth 回调)
11	57	const args = plus.runtime.arguments;
12	58	console.log('🚀 [App] onLaunch 启动参数:', args);
13	59	if (args && args.includes('github-callback')) {
14	60	console.log('🔗 [URL Scheme] 检测到 GitHub 回调 (onLaunch)');
15	61	this.handleUrlScheme({ path: args });
16	62	}
17	63	// #endif
18	64	
19	65	// 清理缓存是好习惯, 予以保留
20	66	uni.removeStorageSync('cachedPostList');
21	67	
22	68	// #ifdef APP-PLUS
23	69	console.log('🔄 [热更新] 当前为 APP-PLUS 环境, 准备获取版本信息...');
24	70	try {
25	71	const systemInfo = uni.getSystemInfoSync();
26	72	const appId = systemInfo.appId;
27	73	console.log('📱 [热更新] App ID:', appId);
28	74	
29	75	plus.runtime.getProperty(plus.runtime.appid, function(widgetInfo) {
30	76	console.log('📱 [热更新] plus.runtime.getProperty 回调已执行');
31	77	if (widgetInfo) {
32	78	// 使用 widgetInfo.versionCode 获取版本号
33	79	const appVersion = widgetInfo.versionCode 0;
34	80	const wgtVersion = widgetInfo.version '';
35	81	console.log('📱 [热更新] App 版本号:', appVersion);
36	82	console.log('📱 [热更新] WGT 资源包版本号:', wgtVersion);
37	83	console.log('📱 [热更新] 当前完整版本信息:', {
38	84	appId: appId,
39	85	appVersion: appVersion,
40	86	wgtVersion: wgtVersion
41	87	});
42	88	// 使用自定义热更新逻辑
43	89	try {
44	90	checkAndUpdate({ silent: true, showConfirm: true }).then((result) => {
45	91	console.log('✅ [热更新] 检查完成:', result);
46	92	}).catch((err) => {
47	93	// 区分不同类型的错误, 提供更友好的提示
48	94	const errorMsg = err?.errMsg err?.message String(err);
49	95	if (errorMsg.includes('resource exhausted') errorMsg.includes('Re

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:96 -> App.vue:143

01	96	console.warn('🚩 [热更新] 检查失败: 云服务资源已耗尽, 请稍后重试
02	97	');
03	98	} else {
04	99	console.warn('🚩 [热更新] 检查失败 (已忽略): ', err);
05	100	}
06	101	});
07	102	} catch (e) {
08	103	console.warn('🚩 [热更新] 调用异常 (已忽略): ', e);
09	104	}
10	105	} else {
11	106	console.error('❌ [热更新] 获取 WGT 版本信息失败, 请检查 manifest.json 配置或
12	107	运行环境。');
13	108	}
14	109	});
15	110	} catch (error) {
16	111	console.warn('🚩 [热更新] 读取版本信息失败 (已忽略): ', error);
17	112	}
18	113	// #endif
19	114	
20	115	// #ifdef H5
21	116	console.log('当前为 H5 环境, 跳过热更新检查');
22	117	// #endif
23	118	
24	119	// 【关键修复】在所有操作之前初始化云开发
25	120	console.log('🔗 [App.vue] onLaunch 开始执行');
26	121	if (typeof wx !== 'undefined' && wx.cloud) {
27	122	console.log('🟡 [App.vue] 检测到 wx.cloud, 立即初始化');
28	123	try {
29	124	wx.cloud.init({
30	125	env: 'cloud1-5gb0pbyl400845f5',
31	126	traceUser: true
32	127	});
33	128	console.log('✅ [App.vue] wx.cloud.init() 调用成功');
34	129	
35	130	// 挂载到全局
36	131	this.\$tcb = wx.cloud;
37	132	if (typeof Vue !== 'undefined' && Vue.prototype) {
38	133	Vue.prototype.\$tcb = wx.cloud;
39	134	}
40	135	if (typeof uni !== 'undefined') {
41	136	uni.\$tcb = wx.cloud;
42	137	}
43	138	console.log('✅ [App.vue] wx.cloud 已挂载到全局');
44	139	} catch (error) {
45	140	console.error('❌ [App.vue] wx.cloud 初始化失败: ', error);
46	141	}
47	142	} else {
48	143	console.log('🚩 [App.vue] wx 或 wx.cloud 不存在');
49	144	console.log('typeof wx:', typeof wx);

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:144 -> App.vue:193

01	144	console.log('wx.cloud:', wx.cloud);
02	145	}
03	146	}
04	147	
05	148	// 【性能优化】立即标记登录流程已开始，不阻塞后续操作
06	149	this.globalData._loginProcessStarted = true;
07	150	
08	151	// 【字体预加载】小程序启动时立即下载汇文明朝字体到本地缓存
09	152	// 下载前使用微信默认字体，下载后自动切换到汇文明朝
10	153	// App 和 H5 环境已经打包了本地字体文件，无需下载
11	154	// #ifdef MP-WEIXIN
12	155	console.log('abs [App.vue] 小程序环境，开始预加载汇文明朝字体');
13	156	console.log('abs [App.vue] 字体文件大小: 7.9MB, 预计需要 10-30 秒');
14	157	
15	158	let lastProgress = 0;
16	159	fontManager.ensureFontAvailable('汇文明朝', (progress) => {
17	160	// 每 10% 输出一次日志，避免刷屏
18	161	if (progress - lastProgress >= 10 progress === 100) {
19	162	console.log(`abs [App.vue] 汇文明朝字体加载进度: \${progress}%`);
20	163	lastProgress = progress;
21	164	}
22	165	}).then(() => {
23	166	console.log('✅ [App.vue] 汇文明朝字体预加载完成，已缓存到本地');
24	167	console.log('✅ [App.vue] 后续启动将直接使用缓存，无需重新下载');
25	168	// 字体加载完成后，可以触发全局事件通知页面刷新
26	169	try {
27	170	uni.\$emit && uni.\$emit('font-loaded', { fontFamily: '汇文明朝' });
28	171	} catch (e) {}
29	172	}).catch(err => {
30	173	console.warn('⚠️ [App.vue] 汇文明朝字体预加载失败，将使用系统默认字体:', err);
31	174	console.warn('⚠️ [App.vue] 可能原因: ');
32	175	console.warn(' 1. 网络连接不稳定');
33	176	console.warn(' 2. 域名配置问题（需要配置 request 合法域名）');
34	177	console.warn(' 3. 字体文件较大，加载超时');
35	178	console.warn(' 4. 系统默认字体仍可正常使用');
36	179	});
37	180	// #endif
38	181	
39	182	// 【性能优化】使用 nextTick 延迟执行非关键任务，让页面先渲染
40	183	this.\$nextTick(() => {
41	184	// 执行登录流程（异步，不阻塞UI）
42	185	this.loginAndCheckUser();
43	186	});
44	187	},
45	188	
46	189	// 【新增】onShow 生命周期：处理 App 从后台唤醒的情况
47	190	onShow: function(options) {
48	191	console.log(' [App] onShow 触发');
49	192	

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:194 -> App.vue:243

01	194	// 获取启动参数 (URL Scheme)
02	195	const args = plus.runtime.arguments;
03	196	console.log(' [URL Scheme] 启动参数:', args);
04	197	
05	198	// 如果有 URL Scheme 参数, 则处理
06	199	if (args && args.includes('github-callback')) {
07	200	console.log(' [URL Scheme] 检测到 GitHub 回调');
08	201	this.handleUrlScheme({ path: args });
09	202	}
10	203	
11	204	// 监听新的 intent 事件 (Android 特有)
12	205	// 当 App 已在后台运行时, 通过 URL Scheme 唤起会触发此事件
13	206	// 注意: 避免重复添加监听器
14	207	if (!this._newintentListenerAdded) {
15	208	plus.globalEvent.addEventListener('newintent', (e) => {
16	209	console.log(' [newintent] 收到新的 intent 事件');
17	210	
18	211	// 获取启动参数
19	212	const newArgs = plus.runtime.arguments;
20	213	if (newArgs) {
21	214	console.log(' [newintent] 启动参数:', newArgs);
22	215	// 将参数转换为 options 格式
23	216	this.handleUrlScheme({ path: newArgs });
24	217	}
25	218	});
26	219	this._newintentListenerAdded = true;
27	220	}
28	221	// #endif
29	222	},
30	223	
31	224	// 【重构】3. 将所有方法都放入 methods 对象中, 这是 Vue 的标准做法
32	225	methods: {
33	226	/**
34	227	* 处理 URL Scheme 启动 (GitHub OAuth 回调)
35	228	*/
36	229	handleUrlScheme(options) {
37	230	// #ifdef APP-PLUS
38	231	if (options && options.path) {
39	232	console.log(' [URL Scheme] 检测到 URL Scheme 启动:', options.path);
40	233	
41	234	// 解析 URL: poementer://github-callback?type=register&data=...
42	235	if (options.path.includes('github-callback')) {
43	236	try {
44	237	const url = options.path;
45	238	const params = {};
46	239	const queryString = url.split('?')[1];
47	240	if (queryString) {
48	241	queryString.split('&').forEach(param => {
49	242	const [key, value] = param.split('=');

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:244 -> App.vue:292

01	244	});
02	245	}
03	246	
04	247	const type = params.type; // 'register' 或 'login'
05	248	const data = params.data;
06	249	
07	250	console.log(' [URL Scheme] 回调类型:', type);
08	251	console.log(' [URL Scheme] 回调数据:', data);
09	252	
10	253	if (type === 'register') {
11	254	// 新用户注册: 跳转到注册页面
12	255	console.log(' [GitHub] 新用户, 跳转到注册页面');
13	256	
14	257	// 解析并保存 GitHub 数据到本地存储, 供注册页面使用
15	258	try {
16	259	const githubData = JSON.parse(data);
17	260	uni.setStorageSync('github_temp_data', githubData);
18	261	console.log(' [GitHub] GitHub 数据已保存:', githubData);
19	262	} catch (e) {
20	263	console.error(' [GitHub] 解析 GitHub 数据失败:', e);
21	264	}
22	265	
23	266	uni.showToast({
24	267	title: '欢迎! 请完成注册',
25	268	icon: 'none',
26	269	duration: 2000
27	270	});
28	271	
29	272	// 跳转到注册页面
30	273	setTimeout(() => {
31	274	uni.reLaunch({
32	275	url: `/pages/register/register?fromGithub=true&githubData=\${
33	276	encodeURIComponent(data)}`
34	276	});
35	277	}, 500);
36	278	
37	279	} else if (type === 'login') {
38	280	// 已注册用户登录: 保存用户信息并跳转
39	281	console.log(' [GitHub] 已注册用户, 处理登录');
40	282	
41	283	try {
42	284	const loginData = JSON.parse(data);
43	285	const userInfo = loginData.user;
44	286	
45	287	// 更新全局用户信息
46	288	this.globalData.userInfo = userInfo;
47	289	this.globalData.openid = userInfo._openid userInfo.openid;
48	290	
49	291	// 更新 getApp().globalData

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:293 -> App.vue:339

01	293	if (appInstance) {
02	294	appInstance.globalData = appInstance.globalData {};
03	295	appInstance.globalData.userInfo = userInfo;
04	296	appInstance.globalData.openid = userInfo._openid userInfo
05	297	.openid;
06	298	appInstance.globalData._loginProcessCompleted = true;
07	299	}
08	300	
09	301	// 保存到本地存储
10	302	uni.setStorageSync('userInfo', userInfo);
11	303	uni.setStorageSync('github_access_token', loginData.accessToken)
12	304	;
13	305	
14	306	console.log(' [GitHub] 用户信息已保存:', userInfo);
15	307	
16	308	uni.showToast({
17	309	title: '登录成功! ',
18	310	icon: 'success',
19	311	duration: 2000
20	312	});
21	313	
22	314	// 跳转到诗歌广场
23	315	setTimeout(() => {
24	316	uni.reLaunch({
25	317	url: '/pages/poem-square/poem-square'
26	318	});
27	319	}, 500);
28	320	
29	321	} catch (e) {
30	322	console.error(' [GitHub] 处理登录数据失败:', e);
31	323	uni.showToast({
32	324	title: '登录失败, 请重试',
33	325	icon: 'none'
34	326	});
35	327	}
36	328	} else if (type === 'error') {
37	329	// 错误处理: 显示错误信息
38	330	console.log(' [GitHub] 收到错误回调');
39	331	
40	332	try {
41	333	const errorInfo = JSON.parse(data);
42	334	const errorMessage = errorInfo.message 'GitHub登录失败, 请重
43	335	试';
44	336	
45	337	console.error(' [GitHub] 错误信息:', errorMessage);
46	338	
47	339	uni.showModal({
48		title: 'GitHub 登录失败',
49		content: errorMessage,

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:340 -> App.vue:389

01	340	confirmText: '知道了',
02	341	success: () => {
03	342	// 返回到登录页面
04	343	uni.reLaunch({
05	344	url: '/pages/login/login'
06	345	});
07	346	}
08	347	});
09	348	
10	349	} catch (e) {
11	350	console.error(' [GitHub] 解析错误数据失败:', e);
12	351	uni.showToast({
13	352	title: 'GitHub登录失败',
14	353	icon: 'none',
15	354	duration: 2000
16	355	});
17	356	
18	357	// 返回到登录页面
19	358	setTimeout(() => {
20	359	uni.reLaunch({
21	360	url: '/pages/login/login'
22	361	});
23	362	}, 2000);
24	363	}
25	364	}
26	365	} catch (error) {
27	366	console.error(' [URL Scheme] 解析失败:', error);
28	367	uni.showToast({
29	368	title: '授权失败, 请重试',
30	369	duration: 2000
31	370	});
32	371	}
33	372	}
34	373	}
35	374	// #endif
36	375	},
37	376	
38	377	/**
39	378	* 获取当前页面路径
40	379	*/
41	380	getCurrentPagePath() {
42	381	try {
43	382	const pages = getCurrentPages();
44	383	const currentPage = pages[pages.length - 1];
45	384	if (currentPage) {
46	385	return currentPage.route currentPage.\$page?.fullPath '';
47	386	}
48	387	} catch (e) {
49	388	console.warn('获取当前页面路径失败:', e);

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:390 -> App.vue:437

01	390	return '';
02	391	},
03	392	
04	393	/**
05	394	* 检查是否是允许的页面（现在允许所有页面未登录访问）
06	395	*/
07	396	isAllowedPageForUnauthenticated() {
08	397	// 【修改】允许所有页面未登录访问，只在特定操作时提示登录
09	398	return true;
10	399	},
11	400	
12	401	// 【重构 & 修正】4. 使用 async/await 重写整个登录流程，代码更清晰
13	402	async loginAndCheckUser() {
14	403	// 检查 \$tcb 实例是否存在
15	404	if (!this.\$tcb) {
16	405	console.error('致命错误: this.\$tcb 未定义! 请检查 main.js 的初始化代码是否执行!'
17	406	');
18	407	return; // 中断执行
19	408	}
20	409	
21	410	// 步骤一: 检查本地缓存，但需要验证云端账户
22	411	const cachedUserInfo = uni.getStorageSync('userInfo');
23	412	if (cachedUserInfo && cachedUserInfo._openid) {
24	413	try {
25	414	// 只在 H5/APP 环境进行匿名认证
26	415	// #ifdef H5 APP-PLUS
27	416	const currentUser = this.\$tcb.auth().currentUser;
28	417	if (!currentUser) {
29	418	await this.\$tcb.auth().signInAnonymously();
30	419	}
31	420	// #endif
32	421	
33	422	// 调用云函数验证用户是否存在
34	423	const verifyRes = await this.\$tcb.callFunction({
35	424	name: 'getUserProfile',
36	425	data: { userId: cachedUserInfo._openid }
37	426	});
38	427	
39	428	if (verifyRes.result && verifyRes.result.success && verifyRes.result.userInf
40	429	o) {
41	430	// 使用云端返回的最新用户信息
42	431	const latestUserInfo = verifyRes.result.userInfo;
43	432	
44	433	// 同时更新 this.globalData 和 getApp().globalData
45	434	this.globalData.userInfo = latestUserInfo;
46	435	this.globalData.openid = latestUserInfo._openid;
47	436	
48	437	// 确保 getApp().globalData 也被正确设置
49	438	const appInstance = getApp();

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:438 -> App.vue:486

01	438	appInstance.globalData = appInstance.globalData {};
02	439	appInstance.globalData.userInfo = latestUserInfo;
03	440	appInstance.globalData.openid = latestUserInfo._openid;
04	441	// 【关键修复】设置登录状态标记
05	442	appInstance.globalData.isLoggedIn = true;
06	443	console.log('✅ [登录流程] getApp().globalData 已更新:', appInstance.
07	444	globalData);
08	444	} else {
09	445	}
10	446	
11	447	// 更新本地缓存为最新的用户信息
12	448	uni.setStorageSync('userInfo', latestUserInfo);
13	449	
14	450	// 标记登录流程已完成
15	451	this.globalData._loginProcessCompleted = true;
16	452	if (appInstance) {
17	453	appInstance.globalData._loginProcessCompleted = true;
18	454	}
19	455	
20	456	console.log('✅ [登录流程] 缓存验证成功, 用户已登录');
21	457	return; // 登录成功, 结束流程
22	458	} else {
23	459	console.log('⚠️ [登录流程] 云端验证失败, 用户账户不存在, 将重新注册');
24	460	// 清除无效的缓存
25	461	uni.removeStorageSync('userInfo');
26	462	// 继续执行注册流程
27	463	}
28	464	} catch (error) {
29	465	console.error('❌ [登录流程] 云端验证失败:', error);
30	466	console.log('⚠️ [登录流程] 验证失败, 将重新注册');
31	467	// 清除可能无效的缓存
32	468	uni.removeStorageSync('userInfo');
33	469	// 继续执行注册流程
34	470	}
35	471	}
36	472	
37	473	// 步骤二: 缓存未命中, 执行完整的云端登录
38	474	// 【修改】允许未登录用户访问所有页面, 不强制重定向
39	475	console.log('👁️ [登录流程] 缓存未命中, 但允许未登录访问, 继续后台登录...');
40	476	
41	477	console.log('👁️ [登录流程] 缓存未命中, 开始执行云端登录...');
42	478	
43	479	try {
44	480	// 检查是否已经登录, 避免重复登录 (仅 H5/APP)
45	481	// #ifdef H5 APP-PLUS
46	482	const currentUser = this.\$tcb.auth().currentUser;
47	483	if (!currentUser) {
48	484	console.log('🔒 [认证] 尝试匿名登录...');
49	485	const authResult = await this.\$tcb.auth().signInAnonymously();

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:487 -> App.vue:535

01	487	} else {
02	488	console.log('✅ [认证] 用户已登录, 跳过匿名登录');
03	489	}
04	490	// #endif
05	491	
06	492	// 【修正】调用 this.\$tcb, 而不是 uniCloud!
07	493	const loginRes = await this.\$tcb.callFunction({
08	494	name: 'login' // 调用您在 TCB 中的 login 云函数
09	495	});
10	496	
11	497	// 详细打印login云函数的返回数据, 帮助调试
12	498	console.log('🔍 [调试] login云函数完整返回数据:', loginRes);
13	499	console.log('🔍 [调试] loginRes.result:', loginRes.result);
14	500	console.log('🔍 [调试] loginRes.openid:', loginRes.openid);
15	501	
16	502	// 尝试多种方式获取openid
17	503	let openid = null;
18	504	if (loginRes.result && loginRes.result.openid) {
19	505	openid = loginRes.result.openid;
20	506	console.log('✅ [调试] 从result.openid获取到openid:', openid);
21	507	} else if (loginRes.openid) {
22	508	openid = loginRes.openid;
23	509	console.log('✅ [调试] 从根级别openid获取到openid:', openid);
24	510	} else if (loginRes.result && loginRes.result.uid) {
25	511	openid = loginRes.result.uid;
26	512	console.log('✅ [调试] 从result.uid获取到openid:', openid);
27	513	} else {
28	514	console.error('❌ [登录流程] 无法从login云函数返回数据中获取openid');
29	515	console.error('❌ [登录流程] 完整返回数据:', JSON.stringify(loginRes, null, 2
30	516	));
31	517	throw new Error('云函数 login 未返回 openid');
32	518	}
33	519	console.log('✅ [云函数 login] 调用成功, openid: ', openid);
34	520	this.globalData.openid = openid;
35	521	uni.setStorageSync('userOpenId', openid); // 缓存 openid
36	522	
37	523	// 步骤三: 根据 openid 查询用户数据库
38	524	// 【修正】使用 this.\$tcb.database() 获取数据库实例
39	525	const db = this.\$tcb.database();
40	526	const userRes = await db.collection('users').where({
41	527	_openid: openid
42	528	}).get();
43	529	
44	530	if (userRes.data.length > 0) {
45	531	// 用户已存在, 登录成功
46	532	const userInfo = userRes.data[0];
47	533	console.log('✅ [数据库查询] 用户已注册, 登录成功: ', userInfo);
48	534	this.globalData.userInfo = userInfo;
49	535	uni.setStorageSync('userInfo', userInfo); // 写入缓存

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:536 -> App.vue:584

01	536	// 用户不存在, 是新用户
02	537	console.log('😭 [数据库查询] 新用户, 尚未注册');
03	538	this.globalData.userInfo = null; // 确保 userInfo 为 null
04	539	}
05	540	
06	541	// 无论新旧用户, 都更新 getApp() 的 globalData
07	542	const appInstance = getApp();
08	543	if (appInstance) {
09	544	appInstance.globalData = appInstance.globalData {};
10	545	appInstance.globalData.userInfo = this.globalData.userInfo;
11	546	appInstance.globalData.openid = this.globalData.openid;
12	547	appInstance.globalData._loginProcessCompleted = true; // 标记登录流程已完成
13	548	console.log('✅ [登录流程] getApp().globalData 已更新:', appInstance.globalDa
14	549	ta);
15	549	} else {
16	550	}
17	551	
18	552	} catch (err) {
19	553	// 处理登录错误
20	554	
21	555	// 即使登录失败, 也标记登录流程已完成, 避免后续显示登录提示
22	556	const appInstance = getApp();
23	557	if (appInstance) {
24	558	appInstance.globalData = appInstance.globalData {};
25	559	appInstance.globalData._loginProcessCompleted = true;
26	560	}
27	561	
28	562	uni.showToast({
29	563	icon: 'none',
30	564	title: '登录失败, 请稍后重试'
31	565	});
32	566	}
33	567	},
34	568	
35	569	}
36	570	};
37	571	
38	572	</script>
39	573	
40	574	<style>
41	575	/* 全局字体预加载 - 仅用于诗歌内容 */
42	576	/* 小程序不支持 CSS @font-face 加载本地字体, 只在 H5 和 APP 环境使用 */
43	577	/* #ifndef MP-WEIXIN */
44	578	@font-face {
45	579	font-family: '汇文明朝';
46	580	src: url('/static/fonts/Huiwen-mincho-compressed.woff2') format('woff2');
47	581	font-weight: normal;
48	582	font-style: normal;
49	583	font-display: swap; /* 优化字体加载性能 */

涉及文件: App.vue

回车键Poementer软件

源码范围: App.vue:585 -> App.vue:634

01	585	/* #endif */
02	586	
03	587	/* 全局样式: 修改下拉刷新的loading转圈颜色为黑色 */
04	588	/* 针对微信小程序 */
05	589	/* #ifdef MP-WEIXIN */
06	590	.wx-pull-refresh {
07	591	color: #000000 !important;
08	592	}
09	593	
10	594	.wx-pull-refresh .wx-pull-refresh-spinner {
11	595	color: #000000 !important;
12	596	}
13	597	
14	598	.wx-pull-refresh .wx-pull-refresh-spinner::before {
15	599	color: #000000 !important;
16	600	}
17	601	/* #endif */
18	602	
19	603	/* 针对H5 */
20	604	/* #ifdef H5 */
21	605	.uni-pull-refresh {
22	606	color: #000000 !important;
23	607	}
24	608	
25	609	.uni-pull-refresh .uni-pull-refresh-spinner {
26	610	color: #000000 !important;
27	611	}
28	612	
29	613	.uni-pull-refresh .uni-pull-refresh-spinner::before {
30	614	color: #000000 !important;
31	615	}
32	616	/* #endif */
33	617	
34	618	/* 针对App */
35	619	/* #ifdef APP-PLUS */
36	620	.uni-pull-refresh {
37	621	color: #000000 !important;
38	622	}
39	623	
40	624	.uni-pull-refresh .uni-pull-refresh-spinner {
41	625	color: #000000 !important;
42	626	}
43	627	
44	628	.uni-pull-refresh .uni-pull-refresh-spinner::before {
45	629	color: #000000 !important;
46	630	}
47	631	/* #endif */
48	632	
49	633	/* 通用样式: 修改所有下拉刷新的loading颜色 */

涉及文件: App.vue -> utils/cloudCall.js (共 2 个文件)

回车键Poementer软件

源码范围: App.vue:635 -> utils/cloudCall.js:18

01	635	.wx-pull-refresh,
02	636	.pull-refresh {
03	637	color: #000000 !important;
04	638	}
05	639	
06	640	.uni-pull-refresh .uni-pull-refresh-spinner,
07	641	.wx-pull-refresh .wx-pull-refresh-spinner,
08	642	.pull-refresh .pull-refresh-spinner {
09	643	color: #000000 !important;
10	644	border-color: #000000 !important;
11	645	}
12	646	
13	647	.uni-pull-refresh .uni-pull-refresh-spinner::before,
14	648	.wx-pull-refresh .wx-pull-refresh-spinner::before,
15	649	.pull-refresh .pull-refresh-spinner::before {
16	650	color: #000000 !important;
17	651	border-color: #000000 !important;
18	652	}
19	653	
20	654	/* 修改下拉刷新指示器的颜色 */
21	655	.uni-pull-refresh-indicator,
22	656	.wx-pull-refresh-indicator {
23	657	color: #000000 !important;
24	658	}
25	659	
26	660	.uni-pull-refresh-indicator .uni-pull-refresh-spinner,
27	661	.wx-pull-refresh-indicator .wx-pull-refresh-spinner {
28	662	color: #000000 !important;
29	663	border-color: #000000 !important;
30	664	}
31	665	</style>
32	FILE FILE 03 utils/cloudCall.js 统一云函数调用封装	
33	1	const platformDetector = require('./platformDetector.js');
34	2	const auth = require('./auth.js');
35	3	
36	4	const DEFAULT_RETRY_DELAY = 300;
37	5	
38	6	function ensureObject(value) {
39	7	if (!value typeof value !== 'object') {
40	8	return {};
41	9	}
42	10	if (Array.isArray(value)) {
43	11	return value.slice();
44	12	}
45	13	return Object.assign({}, value);
46	14	}
47	15	
48	16	function createError(code, message, originalError) {
49	17	const error = originalError instanceof Error ? originalError : new Error(message code);

涉及文件: utils/cloudCall.js

回车键Poementer软件

源码范围: utils/cloudCall.js:19 -> utils/cloudCall.js:67

01	19	if (originalError && originalError !== error) {
02	20	error.originalError = originalError;
03	21	}
04	22	return error;
05	23	}
06	24	
07	25	function getTcbInstance(context) {
08	26	if (context && context.\$tcb && typeof context.\$tcb.callFunction === 'function') {
09	27	return context.\$tcb;
10	28	}
11	29	if (typeof getApp === 'function') {
12	30	const app = getApp();
13	31	if (app && app.\$tcb && typeof app.\$tcb.callFunction === 'function') {
14	32	return app.\$tcb;
15	33	}
16	34	}
17	35	if (typeof uni !== 'undefined' && uni.\$tcb && typeof uni.\$tcb.callFunction === 'function') {
18	36	return uni.\$tcb;
19	37	}
20	38	return null;
21	39	}
22	40	
23	41	async function invokeCloudFunction(method, payload) {
24	42	console.log(`[invokeCloudFunction] 调用方式: \${method}`);
25	43	
26	44	if (method === 'tcb') {
27	45	const instance = getTcbInstance(payload.context);
28	46	console.log(`[invokeCloudFunction] TCB实例:`, instance ? '可用' : '不可用');
29	47	if (!instance) {
30	48	throw createError('TCB_NOT_AVAILABLE', 'TCB 实例不可用');
31	49	}
32	50	return instance.callFunction(payload.options, undefined, payload.customReqOpts);
33	51	}
34	52	
35	53	if (method === 'wx-cloud') {
36	54	console.log(`[invokeCloudFunction] 使用 wx.cloud 调用云函数`);
37	55	if (typeof wx === 'undefined' !wx.cloud) {
38	56	throw createError('WX_CLOUD_NOT_AVAILABLE', 'wx.cloud 不可用');
39	57	}
40	58	
41	59	console.log(`[invokeCloudFunction] wx.cloud 可用, 调用云函数:`, payload.options.name)
42	60	;
43	60	return wx.cloud.callFunction(payload.options);
44	61	}
45	62	
46	63	if (typeof uniCloud !== 'undefined' && typeof uniCloud.callFunction === 'function') {
47	64	return uniCloud.callFunction(payload.options);
48	65	}
49	66	

涉及文件: utils/cloudCall.js

回车键Poementer软件

源码范围: utils/cloudCall.js:68 -> utils/cloudCall.js:116

01	68	}
02	69	
03	70	async function cloudCall(name, data = {}, options = {}) {
04	71	if (!name typeof name !== 'string') {
05	72	return Promise.reject(createError('INVALID_NAME', '云函数名称无效'));
06	73	}
07	74	
08	75	const {
09	76	pageTag = 'global',
10	77	retry = 0,
11	78	retryDelay = DEFAULT_RETRY_DELAY,
12	79	context,
13	80	injectOpenId,
14	81	requireAuth = false,
15	82	timeoutMs
16	83	} = options;
17	84	const shouldInjectOpenId = typeof injectOpenId === 'boolean' ? injectOpenId : !['login', 'ge
18	85	tOpenId', 'github-auth'].includes(name);
19	85	
20	86	const payload = ensureObject(data);
21	87	let openid = null;
22	88	
23	89	if (shouldInjectOpenId) {
24	90	try {
25	91	openid = await auth.getOpenId();
26	92	} catch (error) {
27	93	console.error(`[cloudCall][\${pageTag}] 获取 openid 失败`, error);
28	94	throw createError('NO_OPENID', '获取 openid 失败', error);
29	95	}
30	96	
31	97	if (!openid) {
32	98	if (requireAuth) {
33	99	const error = createError('NO_OPENID', '用户未登录或 openid 缺失');
34	100	
35	101	// 使用统一的登录提示
36	102	if (typeof uni !== 'undefined') {
37	103	try {
38	104	// 动态导入authHelper, 避免循环依赖
39	105	const authHelper = await import('./authHelper.js');
40	106	await authHelper.requireLogin({
41	107	content: '此操作需要登录, 请先登录'
42	108	});
43	109	} catch (importError) {
44	110	// 降级到简单的toast提示
45	111	if (uni.showToast) {
46	112	uni.showToast({
47	113	title: '请先登录',
48	114	icon: 'none'
49	115	});

涉及文件: utils/cloudCall.js

回车键Poementer软件

源码范围: utils/cloudCall.js:117 -> utils/cloudCall.js:162

01	117	}
02	118	}
03	119	
04	120	console.warn(`[cloudCall][\${pageTag}] openid 缺失, 调用 "\${name}" 失败`);
05	121	throw error;
06	122	}
07	123	console.warn(`[cloudCall][\${pageTag}] openid 缺失, 调用 "\${name}" 将不注入 openid`);
08	124	} else if (!payload.openid) {
09	125	payload.openid = openid;
10	126	}
11	127	}
12	128	
13	129	const method = platformDetector.getCloudFunctionMethod();
14	130	console.log(`🔗 [cloudCall] 云函数调用方式: \${method}, 函数名: \${name}`);
15	131	const totalAttempts = Math.max(0, parseInt(retry, 10)) + 1;
16	132	
17	133	for (let attempt = 1; attempt <= totalAttempts; attempt += 1) {
18	134	try {
19	135	console.log(`[cloudCall][\${pageTag}] 调用云函数 "\${name}" (尝试 \${attempt}/\${totalAt
20	136	tempts)`), payload);
21	136	const result = await invokeCloudFunction(method, {
22	137	context,
23	138	options: {
24	139	name,
25	140	data: payload
26	141	},
27	142	customReqOpts: typeof timeoutMs === 'number' && timeoutMs > 0 ? { timeout: timeo
28	143	utMs } : undefined
29	143	});
30	144	console.log(`[cloudCall][\${pageTag}] 云函数 "\${name}" 调用成功`, result);
31	145	return result;
32	146	} catch (error) {
33	147	const isLastAttempt = attempt === totalAttempts;
34	148	const errorCode = error && error.code ? error.code : error?.errCode;
35	149	
36	150	console.error(`[cloudCall][\${pageTag}] 云函数 "\${name}" 调用失败 (尝试 \${attempt}/\${
37	151	totalAttempts)`), error);
38	151	
39	152	if (errorCode === 'NO_OPENID') {
40	153	throw createError('NO_OPENID', '用户未登录或 openid 缺失', error);
41	154	}
42	155	
43	156	if (isLastAttempt) {
44	157	const finalError = createError(errorCode 'CLOUD_CALL_FAILED', '云函数调用失败
45	158	`, error);
46	158	if (typeof uni !== 'undefined' && uni.showToast) {
47	159	uni.showToast({
48	160	title: '网络异常, 请稍后再试',
49	161	icon: 'none'

涉及文件: utils/cloudCall.js -> api-cache/_shared/cloud-wrapper.js (共 2 个文件)

回车键Poementer软件

源码范围: utils/cloudCall.js:163 -> api-cache/_shared/cloud-wrapper.js:33

01	163	}
02	164	throw finalError;
03	165	}
04	166	
05	167	if (retryDelay > 0) {
06	168	await new Promise((resolve) => setTimeout(resolve, retryDelay));
07	169	}
08	170	}
09	171	}
10	172	
11	173	throw createError('CLOUD_CALL_FAILED', '云函数调用失败');
12	174	}
13	175	
14	176	module.exports = {
15	177	cloudCall
16	178	};
17	FILE FILE 04 api-cache/_shared/cloud-wrapper.js 语义 API 解包与错误归一	
18	1	const { cloudCall } = require('../utils/cloudCall.js');
19	2	
20	3	function getResult(res) {
21	4	return (res && res.result) {};
22	5	}
23	6	
24	7	function isSuccessResult(result) {
25	8	return !result && (result.success === true result.code === 0);
26	9	}
27	10	
28	11	function unwrapResult(res, fallbackMessage = '操作失败') {
29	12	const result = getResult(res);
30	13	if (!isSuccessResult(result)) {
31	14	throw new Error(result.error result.message result.msg fallbackMessage);
32	15	}
33	16	return result;
34	17	}
35	18	
36	19	async function callCloudAndUnwrap(name, payload = {}, options = {}, fallbackMessage) {
37	20	const res = await cloudCall(name, payload, options);
38	21	return unwrapResult(res, fallbackMessage);
39	22	}
40	23	
41	24	async function callCloudAndGetResult(name, payload = {}, options = {}) {
42	25	const res = await cloudCall(name, payload, options);
43	26	return getResult(res);
44	27	}
45	28	
46	29	async function callActionAndUnwrap({
47	30	functionName,
48	31	action,
49	32	payload = {},

涉及文件: api-cache/_shared/cloud-wrapper.js

回车键Poementer软件

源码范围: api-cache/_shared/cloud-wrapper.js:34 -> api-cache/_shared/cloud-wrapper.js:83

01	34	context,
02	35	requireAuth = true,
03	36	fallbackMessage = '操作失败'
04	37	} = {}) {
05	38	return callCloudAndUnwrap(06 39 functionName, 07 40 { 08 41 action, 09 42 ...payload 10 43 }, 11 44 { 12 45 pageTag: pageTag `\${functionName}:\${action}`, 13 46 context, 14 47 requireAuth 15 48 }, 16 49 fallbackMessage 17 50); 18 51 } 19 52 20 53 function createActionCaller({ 21 54 functionName, 22 55 pageTagPrefix, 23 56 requireAuth = true, 24 57 defaultFallbackMessage = '操作失败' 25 58 } = {}) { 26 59 return async function callAction(action, payload = {}, options = {}) { 27 60 const { 28 61 pageTag, 29 62 context, 30 63 fallbackMessage = defaultFallbackMessage, 31 64 requireAuth: required = requireAuth 32 65 } = options; 33 66 return callActionAndUnwrap({ 34 67 functionName, 35 68 action, 36 69 payload, 37 70 pageTag: pageTag `\${pageTagPrefix    functionName}:\${action}`, 38 71 context, 39 72 requireAuth: required, 40 73 fallbackMessage 41 74 }); 42 75 }; 43 76 } 44 77 45 78 module.exports = { 46 79 getResult, 47 80 isSuccessResult, 48 81 unwrapResult, 49 82 callCloudAndGetResult,

涉及文件: api-cache/_shared/cloud-wrapper.js -> functions/login/index.js (共 2 个文件) 回车键Poementer软件

源码范围: api-cache/_shared/cloud-wrapper.js:84 -> functions/login/index.js:46

01	84	callActionAndUnwrap,
02	85	createActionCaller
03	86	};
04	FILE 05 functions/login/index.js 登录云函数入口	
05	1	// 云函数入口文件 - 兼容微信云开发和CloudBase
06	2	const cloud = require('wx-server-sdk');
07	3	
08	4	cloud.init({
09	5	env: cloud.DYNAMIC_CURRENT_ENV
10	6	});
11	7	
12	8	/**
13	9	* 通用login函数, 处理来自小程序、App、Web的调用
14	10	*/
15	11	exports.main = async (event, context) => {
16	12	try {
17	13	console.log(' [云函数] login被调用, event:', event);
18	14	console.log(' [云函数] login被调用, context:', context);
19	15	
20	16	const wxContext = cloud.getWXContext();
21	17	console.log(' [云函数] wxContext:', wxContext);
22	18	
23	19	// 在H5环境下, wxContext.OPENID可能为空, 使用context中的信息
24	20	const openid = wxContext.OPENID context.OPENID 'anonymous_' + Date.now();
25	21	const appid = wxContext.APPID context.APPID 'unknown';
26	22	const unionid = wxContext.UNIONID context.UNIONID null;
27	23	
28	24	console.log(' [云函数] 最终openid:', openid);
29	25	
30	26	// 返回兼容格式, 同时支持微信云开发和CloudBase
31	27	const result = {
32	28	// 微信云开发格式 (小程序兼容)
33	29	event,
34	30	openid: openid,
35	31	appid: appid,
36	32	unionid: unionid,
37	33	
38	34	// CloudBase格式 (H5兼容)
39	35	success: true,
40	36	uid: openid, // 使用openid作为uid
41	37	result: {
42	38	openid: openid,
43	39	uid: openid,
44	40	appid: appid,
45	41	unionid: unionid
46	42	}
47	43	};
48	44	
49	45	console.log(' [云函数] 返回结果:', result);

涉及文件: functions/contentCheck/index.js

回车键Poementer软件

源码范围: functions/contentCheck/index.js:670 -> functions/contentCheck/index.js:717

01	670	console.log('嫋嫋嫋嫋版嶠拏攪綽綽?..');
02	671	const testResult = await db.collection('posts').limit(1).get();
03	672	console.log('嫋版嶠拏攪綽綽 ヲ 甯革紆嫋嫋嫋嫋 ヲ 緇撤濼:', testResult);
04	673	} catch (testError) {
05	674	console.error('嫋版嶠拏攪綽綽 ヲ 緇撤濼ヶ路?', testError);
06	675	throw new Error(`嫋版嶠拏攪綽綽 ヲ ヶ路? \${testError.message}`);
07	676	}
08	677	
09	678	const result = await db.collection('posts').add({
10	679	data: postData
11	680	});
12	681	
13	682	// 鍬始啓嫋海姪緇燭 嫋拏錫錫規椿鐸 厶 筈鋤拏敗鋤峰弭涓座椿鐸 級
14	683	const targetActivityId = activityId joinActivityId;
15	684	if (targetActivityId) {
16	685	try {
17	686	await db.collection('activities').doc(targetActivityId).update({
18	687	data: {
19	688	postCount: _.inc(1),
20	689	lastPostTime: new Date(),
21	690	updatedAt: new Date()
22	691	}
23	692	});
24	693	} catch (activityUpdateError) {
25	694	console.warn('[contentCheck] 嫋海姪緇燭 嫋拏存枵澶辨触拏棟笱寨卞揷鑒敗戔拏?', activityUpda
26	695	teError);
27	696	}
28	697	
29	698	// 枵藉姑鍬櫛媛拏承 級?植樺厓鑒 ヲ 紙涓庈幫鍾父社変佃B論 紆閭垮厓鍬x増鍾 璞 \$ 稻鍋勦奖鍋櫛級
30	699	try {
31	700	await db.collection('posts').doc(result._id).update({
32	701	data: {
33	702	backgroundColor: backgroundColor postData.backgroundColor '',
34	703	textColor: textColor postData.textColor '#000000',
35	704	highlightSentence: highlightSentenceValue postData.highlightSentence ''
36	705	}
37	706	});
38	707	
39	708	// 鐸煎 鍬櫛媛拏承珮鐸爕 嫋拏拏
40	709	try {
41	710	await db.collection('posts').doc(result._id).update({
42	711	data: {
43	712	highlightLines: Array.isArray(effectiveHighlightLines) ? effectiveHighlightLines : [
44	713	]
45	714	});
46	715	} catch (e) {
47	716	console.warn('[contentCheck] 鍬櫛媛 highlightLines 澶辨触拏拏拷鐸 ヲ 級:', e);

涉及文件: functions/contentCheck/index.js

回车键Poementer软件

源码范围: functions/contentCheck/index.js:718 -> functions/contentCheck/index.js:766

01	718	} catch (e) {
02	719	console.warn('[contentCheck] 枓藉姑鋤穰緩極涸壞/植樺厓鑿 ャヶ璫ワ絰涇整獎鐳整戔甯?', e);
03	720	}
04	721	
05	722	console.log('鑲版嵯寨嶺啓緯 ャ垚鐳燵絰枓旂湢緇嶺灘:', {
06	723	postId: result._id,
07	724	insertedCount: result.stats?.inserted 1
08	725	});
09	726	
10	727	// ----- 4. 闡始師錄沆癡姝劣嚙鐳厶垙寤鴻癡浜轰富樑?-----
11	728	if ((publishMode === 'poem' isSeries) && !isOriginal && authorName && authorName.trim())
12	729	{
13	730	try {
14	731	const poetName = authorName.trim();
15	732	console.log('妨€鐳ヨ癡浜轰富樑垫嶺錫一珣鐳?', poetName);
16	733	
17	734	// 鐳ヨ癡櫛汉鐳櫛愼宸插珣鐳?
18	735	const poetResult = await db.collection('poets')
19	736	.where({ name: poetName })
20	737	.limit(1)
21	738	.get();
22	739	
23	740	if (!poetResult.data poetResult.data.length === 0) {
24	741	// 璇櫛汉涓涇珣鐳櫛絰鐳櫛姝絳浼?
25	742	console.log('璇櫛汉涓涇>涓涇珣鐳櫛絰鐳櫛姝絳浼:', poetName);
26	743	const newPoet = {
27	744	name: poetName,
28	745	avatar: '', // 榛櫛櫛鐳犮?鍡?
29	746	bio: '', // 榛櫛櫛鐳犺岋?
30	747	createTime: db.serverDate(),
31	748	updateTime: db.serverDate(),
32	749	creatorOpenid: currentOpenid // 聰板綽璋恴櫛€鑲戔戔垙?
33	750	};
34	751	
35	752	const poetAddResult = await db.collection('poets').add({ data: newPoet });
36	753	console.log('璇櫛汉涓涇>絳浼����恴恴:', poetAddResult._id);
37	754	} else {
38	755	console.log('璇櫛汉涓涇>宸插珣珣鐳櫛絰鐳犨絳浼:', poetName);
39	756	}
40	757	} catch (poetError) {
41	758	// 璇櫛汉涓涇>絳浼����垙涓涇整獎鐳整戔甯?
42	759	console.warn('鐳櫛姝絳浼璇櫛汉涓涇>湮辯触脍棟笌寨卜楨鑲戔戔戔?', poetError);
43	760	}
44	761	}
45	762	
46	763	// 緯厶儀總恴恴脍劣緇鍡�垚垚鐳�姣?
47	764	return buildSuccess({
48	765	postId: result._id,
49	766	msg: '发布成功'

涉及文件: functions/contentCheck/index.js -> functions/createPost/index.js (共 2 个文件) 回车键Poementer软件

源码范围: functions/contentCheck/index.js:767 -> functions/createPost/index.js:29

01	767	
02	768	} catch (dbError) {
03	769	console.error("鑼版崑擻嶧啓緯 ュ け 璐?", dbError);
04	770	console.error("鑼版崑擻撻枝璇 璦 鄧?", {
05	771	message: dbError.message,
06	772	code: dbError.code,
07	773	stack: dbError.stack
08	774	});
09	775	return buildFailure({
10	776	code: -3,
11	777	msg: '数据库存储失败: ' + dbError.message,
12	778	errorCode: dbError.code 'DB_WRITE_FAILED',
13	779	extra: { error: dbError.message }
14	780	});
15	781	}
16	782	};
17	783	
18	784	
19	785	
20	786	
21	FILE FILE 12 functions/createPost/index.js 创建帖子云函数	
22	1	// 云函数: createPost
23	2	// 功能: 创建新帖子
24	3	
25	4	const cloud = require('wx-server-sdk');
26	5	
27	6	cloud.init({
28	7	env: cloud.DYNAMIC_CURRENT_ENV
29	8	});
30	9	
31	10	const db = cloud.database();
32	11	
33	12	exports.main = async (event, context) => {
34	13	console.log('🔗 [createPost] 开始创建帖子');
35	14	console.log('🔗 [createPost] 请求参数:', JSON.stringify(event, null, 2));
36	15	
37	16	const wxContext = cloud.getWXContext();
38	17	const openid = event.openid wxContext.OPENID;
39	18	
40	19	if (!openid) {
41	20	return {
42	21	code: -1,
43	22	msg: '无法获取用户信息, 请重新登录'
44	23	};
45	24	}
46	25	
47	26	try {
48	27	// 获取用户信息
49	28	const userResult = await db.collection('users').where({

提交页: 34 / 60

回车键Poementer软件

完整序列页: 259 / 285

涉及文件: functions/createPost/index.js

源码范围: functions/createPost/index.js:30 -> functions/createPost/index.js:79

01	30	}).get();
02	31	
03	32	if (userResult.data.length === 0) {
04	33	return {
05	34	code: -1,
06	35	msg: '用户信息不存在'
07	36	};
08	37	}
09	38	
10	39	const user = userResult.data[0];
11	40	
12	41	// 准备帖子数据
13	42	const postData = {
14	43	_openid: openid,
15	44	authorName: event.isAnonymous ? event.anonymousAuthorName : user.nickName,
16	45	authorAvatar: event.isAnonymous ? '/static/images/avatar.png' : user.avatarUrl,
17	46	title: event.title '',
18	47	content: event.content '',
19	48	createTime: db.serverDate(),
20	49	votes: 0,
21	50	commentCount: 0,
22	51	viewCount: 0,
23	52	isPoem: event.publishMode === 'poem' event.isSeries false,
24	53	isSeries: event.isSeries false,
25	54	isOriginal: event.isOriginal false,
26	55	isDiscussion: event.isDiscussion false,
27	56	author: event.author '',
28	57	tags: event.tags [],
29	58	isAnonymous: event.isAnonymous false,
30	59	anonymousAuthorName: event.anonymousAuthorName '匿名用户',
31	60	realAuthorOpenid: event.realAuthorOpenid null
32	61	};
33	62	
34	63	// 添加颜色信息
35	64	if (event.backgroundColor) {
36	65	postData.backgroundColor = event.backgroundColor;
37	66	}
38	67	if (event.textColor) {
39	68	postData.textColor = event.textColor;
40	69	}
41	70	
42	71	// 添加高光行信息
43	72	if (event.highlightLines && event.highlightLines.length > 0) {
44	73	postData.highlightLines = event.highlightLines;
45	74	postData.highlightSentence = event.highlightLines[0];
46	75	}
47	76	
48	77	// 添加图片信息
49	78	if (event.fileIDs && event.fileIDs.length > 0) {

涉及文件: functions/createPost/index.js -> functions/getPostList/index.js (共 2 个文件) 回车键Poementer软件

源码范围: functions/createPost/index.js:80 -> functions/getPostList/index.js:9

01	80	postData.imageUrls = event.fileIDs;
02	81	}
03	82	if (event.originalFileIDs && event.originalFileIDs.length > 0) {
04	83	postData.originalImageUrl = event.originalFileIDs[0];
05	84	postData.originalImageUrls = event.originalFileIDs;
06	85	}
07	86	
08	87	// 讨论模式特殊处理
09	88	if (event.isDiscussion) {
10	89	postData.sentenceGroups = event.sentenceGroups [];
11	90	postData.discussionSentences = event.discussionSentences [];
12	91	}
13	92	
14	93	// 组诗模式特殊处理
15	94	if (event.isSeries) {
16	95	postData.seriesBlocks = event.seriesBlocks [];
17	96	postData.seriesBlockCount = (event.seriesBlocks []).length;
18	97	}
19	98	
20	99	// 创建帖子
21	100	const result = await db.collection('posts').add({
22	101	data: postData
23	102	});
24	103	
25	104	console.log('✅ [createPost] 帖子创建成功:', result._id);
26	105	
27	106	return {
28	107	code: 0,
29	108	msg: '发布成功',
30	109	postId: result._id
31	110	};
32	111	
33	112	} catch (error) {
34	113	console.error('❌ [createPost] 创建帖子失败:', error);
35	114	return {
36	115	code: -1,
37	116	msg: error.message '发布失败'
38	117	};
39	118	}
40	119	};
41	FILE FILE 13 functions/getPostList/index.js 帖子列表云函数	
42	1	// 浜賤𪔐𪔐板𪔐𪔐 f 构浔?
43	2	const cloud = require('wx-server-sdk');
44	3	
45	4	cloud.init({
46	5	env: cloud.DYNAMIC_CURRENT_ENV
47	6	});
48	7	
49	8	const db = cloud.database();

01	10	const \$ = __.aggregate;
02	11	
03	12	// 極戡崑闕愬埜間嶇疆
04	13	const RATE_LIMITS = {
05	14	perMinute: 20, // 姣忔培闕?0娆?
06	15	perHour: 200, // 姣�罐鏷?00娆?
07	16	perDay: 600 // 姣�?600娆?
08	17	};
09	18	
10	19	// 極戡崑闕愬埜?鏷?嚟鏷?
11	20	async function checkRateLimit(openid) {
12	21	try {
13	22	const now = Date.now();
14	23	const minuteAgo = now - 60 * 1000;
15	24	const hourAgo = now - 60 * 60 * 1000;
16	25	const dayAgo = now - 24 * 60 * 60 * 1000;
17	26	
18	27	console.log('鏷? [RateLimit] ?灑?灑?鏷?鏷?鏷?鏷?- openid:', openid.substring(0, 8) + '..
19	28	.');
20	29	
21	30	// ?灑?鏷?培闕愬?闕愬埜
22	31	const minuteCount = await db.collection('api_rate_limits')
23	32	.where({
24	33	openid: openid,
25	34	api_name: 'getPostList',
26	35	timestamp: __.gte(minuteAgo)
27	36	})
28	37	.count();
29	38	
30	39	if (minuteCount.total >= RATE_LIMITS.perMinute) {
31	40	console.log('鏷? [RateLimit] 鏷?灑?灑?灑?灑?灑?灑?', minuteCount.total, '/', RATE_LIMITS.p
32	41	erMinute);
33	42	return {
34	43	allowed: false,
35	44	reason: 'RATE_LIMIT_MINUTE_EXCEEDED',
36	45	resetTime: minuteAgo + 60000,
37	46	limit: RATE_LIMITS.perMinute,
38	47	current: minuteCount.total,
39	48	waitSeconds: Math.ceil((minuteAgo + 60000 - now) / 1000)
40	49	};
41	50	}
42	51	
43	52	// ?灑?鏷?罐鏷?灑?闕愬埜
44	53	const hourCount = await db.collection('api_rate_limits')
45	54	.where({
46	55	openid: openid,
47	56	api_name: 'getPostList',
48	57	timestamp: __.gte(hourAgo)
49	58	})

涉及文件: functions/getPostList/index.js

回车键Poementer软件

源码范围: functions/getPostList/index.js:58 -> functions/getPostList/index.js:105

01	58	
02	59	if (hourCount.total >= RATE_LIMITS.perHour) {
03	60	console.log('裸穀 [RateLimit] 灝忔禄缩ㄣ稼録脩稭闕?', hourCount.total, '/', RATE_LIMITS.per
04	61	Hour);
05	62	return {
06	63	allowed: false,
07	64	reason: 'RATE_LIMIT_HOUR_EXCEEDED',
08	65	resetTime: hourAgo + 3600000,
09	66	limit: RATE_LIMITS.perHour,
10	67	current: hourCount.total,
11	68	waitSeconds: Math.ceil((hourAgo + 3600000 - now) / 1000)
12	69	};
13	70	}
14	71	
15	72	// 姊€鎔ㄣ欄缩ㄣ稼録?
16	73	const dayCount = await db.collection('api_rate_limits')
17	74	.where({
18	75	openid: openid,
19	76	api_name: 'getPostList',
20	77	timestamp: _.gte(dayAgo)
21	78	})
22	79	.count());
23	80	
24	81	if (dayCount.total >= RATE_LIMITS.perDay) {
25	82	console.log('裸穀 [RateLimit] 鐮ㄣ駭闕愬埆瓓响稼:', dayCount.total, '/', RATE_LIMITS.perDa
26	83	y);
27	84	return {
28	85	allowed: false,
29	86	reason: 'RATE_LIMIT_DAY_EXCEEDED',
30	87	resetTime: dayAgo + 86400000,
31	88	limit: RATE_LIMITS.perDay,
32	89	current: dayCount.total,
33	90	waitSeconds: Math.ceil((dayAgo + 86400000 - now) / 1000)
34	91	};
35	92	}
36	93	
37	94	// 璁板綰鐮ㄣ駭鋒脛
38	95	await db.collection('api_rate_limits').add({
39	96	data: {
40	97	openid: openid,
41	98	api_name: 'getPostList',
42	99	timestamp: now
43	100	}
44	101	});
45	102	
46	103	// 淪氣渦妮呼慙杓困渦璁板綰脛?%似偃毘錄ㄣ璁?
47	104	if (Math.random() < 0.01) {
48		try {
49		await db.collection('api_rate_limits')

涉及文件: functions/getPostList/index.js

回车键Poementer软件

源码范围: functions/getPostList/index.js:106 -> functions/getPostList/index.js:153

01	106	timestamp: _.lt(dayAgo)
02	107	)}
03	108	.remove();
04	109	console.log('裸4 [RateLimit] 妮呼恁杓困渴璁板綽淪岫垚');
05	110	} catch (cleanupError) {
06	111	console.warn('鋤狂笏 [RateLimit] 妮呼恁杓困渴璁板綽澶辨触:', cleanupError);
07	112	}
08	113	}
09	114	
10	115	console.log('鈡?[RateLimit] 極戠毘舛€鏢ヲ€氍 - 褰嶷鐳錄嘛挖:', minuteCount.total + 1, '灝
11	116	忔祿:', hourCount.total + 1, '鏹?', dayCount.total + 1);
12	117	return {
13	118	allowed: true,
14	119	currentCounts: {
15	120	perMinute: minuteCount.total + 1,
16	121	perHour: hourCount.total + 1,
17	122	perDay: dayCount.total + 1
18	123	}
19	124	};
20	125	
21	126	} catch (error) {
22	127	console.error('鈹?[RateLimit] 極戠毘閻愶埒舛€鏢 ュけ璐?', error);
23	128	// 舛€鏢 ュけ璐ヲ祿鐳飲閻氍繡铸岢位鐳莖奖鍋嘈甯哥戢鋤?
24	129	return { allowed: true };
25	130	}
26	131	}
27	132	
28	133	// 浜戱噉鏤板瓔鎼 e 噉鏤?
29	134	exports.main = async (event, context) => {
30	135	console.log('裸翅 [getPostList] ===== 浜戱噉鏤板紆濮嫫墩琛?=====');
31	136	console.log('裸翅 [getPostList] 鋤ヲ敕鐳扮宛鎼俗噉:', JSON.stringify(event, null, 2));
32	137	
33	138	const wxContext = cloud.getWXContext();
34	139	const wxCtxOpenid = wxContext.OPENID;
35	140	const eventOpenid = event.openid;
36	141	const openid = eventOpenid wxCtxOpenid;
37	142	// 添加 author 参数用于诗人筛选
38	143	const { skip = 0, limit = 10, isPoem, isOriginal, isDiscussion, tag = '', author = '', include
39	144	Activity = false, activityId = '' } = event;
40	145	const safeActivityId = typeof activityId === 'string' ? activityId.trim() : '';
41	146	const isActivityQuery = includeActivity === true && !!safeActivityId;
42	147	
43	148	console.log('裸翅 [getPostList] 璫f 漣鎼俗噉:', {
44	149	eventOpenid: eventOpenid ? 'provided' : 'missing',
45	150	wxCtxOpenid: wxCtxOpenid ? 'provided' : 'missing',
46	151	chosenOpenidSource: eventOpenid ? 'event.openid' : 'wxContext.OPENID',
47	152	chosenOpenidExists: !!openid,
48		skip,
49		limit,

涉及文件: functions/getPostList/index.js

回车键Poementer软件

源码范围: functions/getPostList/index.js:154 -> functions/getPostList/index.js:203

01	154	isOriginal,
02	155	isDiscussion,
03	156	tag,
04	157	author,
05	158	includeActivity,
06	159	activityId: safeActivityId
07	160	});
08	161	
09	162	if (!openid) {
10	163	console.error('缺少[getPostList] 鉴狗磅鑾峰璩璿儿垲 openid');
11	164	return {
12	165	success: false,
13	166	message: '无法获取用户 openid, 请重新登录',
14	167	code: 'NO_OPENID'
15	168	};
16	169	}
17	170	
18	171	if (includeActivity === true && !safeActivityId) {
19	172	return {
20	173	success: false,
21	174	message: 'activityId 不能为空',
22	175	code: 'INVALID_ACTIVITY_ID'
23	176	};
24	177	}
25	178	
26	179	// 奶€鎔ヨ硃佬硃鑿回椽錄詆紙涓存禄绂佺馭浹ユ师鎔ヨ稭鑿璽棟極梳級
27	180	// TODO: 浣樺寔極戠毘闌愶埗奶€鎔ラ€昏總錫序噸編板碁鑿?
28	181	/*
29	182	const rateLimitResult = await checkRateLimit(openid);
30	183	if (!rateLimitResult.allowed) {
31	184	console.log('裸穀 [getPostList] 極戠毘闌愶埗鎔佅墮:', rateLimitResult.reason);
32	185	return {
33	186	success: false,
34	187	code: rateLimitResult.reason,
35	188	message: `璇鋒脛杙困錄極戠筭铸劣硃鑿?{rateLimitResult.waitSeconds}绉拱愧闕蔣瘥`,
36	189	data: {
37	190	limit: rateLimitResult.limit,
38	191	current: rateLimitResult.current,
39	192	resetTime: rateLimitResult.resetTime,
40	193	waitSeconds: rateLimitResult.waitSeconds
41	194	}
42	195	};
43	196	}
44	197	*/
45	198	
46	199	try {
47	200	console.log('裸邇 [getPostList] 寮€濮嬫潒寔烘燧璿?');
48	201	
49	202	// 鑾峰璩琚硃皖鑿界琒璿儿垲ID錄樁〃铸堞娇璿丨紀瀛梳級

01	204	try {
02	205	const getBlockedUserIds = require('../_lib/get-blocked-user-ids');
03	206	blockedUserIds = await getBlockedUserIds(openid, db);
04	207	console.log('傈嫹 [getPostList] 琚畹琺界苑璽儿垠鐓伴嘶:', blockedUserIds.length);
05	208	} catch (blockError) {
06	209	console.error('鈑?[getPostList] 鑾峰彌瀟他馭錄編 " 澶辨舐:', blockError);
07	210	}
08	211	
09	212	let query = db.collection('posts').aggregate();
10	213	
11	214	// 鏼勻缓绛湢€爰潄涕?
12	215	const matchConditions = { isHidden: _.neq(true) };
13	216	
14	217	if (isActivityQuery) {
15	218	matchConditions.\$or = [
16	219	{
17	220	isActivityPost: true,
18	221	activityId: safeActivityId
19	222	},
20	223	{
21	224	joinedActivityId: safeActivityId
22	225	}
23	226	];
24	227	} else {
25	228	// 甯歌畹錄編 " 榛禪畹錙掬橫娣悔姘甯棧璫
26	229	matchConditions.isActivityPost = _.neq(true);
27	230	}
28	231	
29	232	// 濡俗灘錡囷轟浜哈sPoem鑾俗噎鋒岫坊鐳玃瘥姝出瓮閩爰潄涕?
30	233	if (isPoem !== undefined) {
31	234	if (isPoem === true) {
32	235	// 鑾畹峇鑾枹瘥姝岨細isPoem 蹇响】涓?true
33	236	matchConditions.isPoem = true;
34	237	} else {
35	238	// 鑾畹峇鑾桷潛璇格靈鋒步sPoem 涓?false 鋁柵笱瀛樴鋒?ne: true 洩氫樞問?false鉅芑ull 鍛
36	239	峤笱瀛樴鋒錄勸肚变礪級
37	239	matchConditions.isPoem = _.neq(true);
38	240	}
39	241	console.log('傈嫹 [getPostList] 娣悔姑isPoem绛湢€爰潄涕?', isPoem);
40	242	}
41	243	
42	244	// 濡俗灘錡囷轟浜哈sOriginal鑾俗噎鋒岫坊鐳玃豺師錄浹瓮閩爰潄涕?
43	245	if (isOriginal !== undefined) {
44	246	if (isOriginal === true) {
45	247	// 鑾畹峇鑾桷師錄空細isOriginal 蹇响】涓?true
46	248	matchConditions.isOriginal = true;
47	249	} else {
48	250	// 鑾畹峇鑾桷潛錒墻挡鋒步sOriginal 涓?false 鋁柵笱瀛樴鋒?ne: true 洩氫樞問?false鉅芑ull
49	251	鍛峤笱瀛樴鋒錄勸肚变礪級

01	252	}
02	253	console.log('裸翅 [getPostList] 娣诿姑isOriginal绎湏€爰潄潄?', isOriginal);
03	254	}
04	255	
05	256	// 濡俗漶鏄囙晶浜哇ag縡俗暝铸岫坊鐳灼燻绎剧岔闾爰潄潄?
06	257	if (tag) {
07	258	matchConditions.tags = tag; // 鋹归麻鋹咄裸璇ㄣ燻绎剧苑鋹困。
08	259	matchConditions['tags.0'] = { \$exists: true }; // 纭ㄣ縡tags鏢扮耗鏄囙肝鏈爰潄涓ㄣ压缁?
09	260	console.log('裸翅 [getPostList] 娣诿姑tag绎湏€爰潄潄?', tag);
10	261	}
11	262	
12	263	// 濡俗漶鏄囙晶浜唵sDiscussion縡俗暝铸岫坊鐳抗ㄣ聰虹岔闾爰潄潄?
13	264	if (isDiscussion !== undefined) {
14	265	if (isDiscussion === true) {
15	266	// 縡ㄣ唵縡梏ㄣ聰猴細isDiscussion 蹇响」涓?true
16	267	matchConditions.isDiscussion = true;
17	268	} else {
18	269	// 縡ㄣ唵縡桤漶聰ㄣㄣ铸步sDiscussion 涓?false 鋹柵笱瀛樺潄铸?ne: true 浼氭蹙問?false鋹蓑ul
19	270	} 1 鍛帆笱瀛樺潄潄鋹勸ㄣ变礪級
20	271	matchConditions.isDiscussion = _.neg(true);
21	272	}
22	273	console.log('裸翅 [getPostList] 娣诿姑isDiscussion绎湏€爰潄潄?', isDiscussion);
23	274	}
24	275	
25	276	// 鏄爰瘥浜猴紙浣涓ㄣ咆級绎湏€?
26	277	if (author && author.trim()) {
27	278	matchConditions.author = author.trim();
28	279	console.log('裸翅 [getPostList] 娣诿姑author绎湏€爰潄潄?', author);
29	280	}
30	281	
31	282	// 枊困护琚ㄣ院鏄囙界馊鋹风殃甯棧璫铸塥健璇(ㄣ)椿变靛所枊困护 _openid铸出栢鏄囙ㄣ鐳曄ㄣ鐳曄椿变靛枊困
32	283	护 realAuthorOpenid铸?
33	284	if (blockedUserIds.length > 0) {
34	285	// 鏄儿健璇(ㄣ)椿变佃繢婊ㄣㄣ縡閫�氥笱瀛怛冤 _openid
35	286	matchConditions._openid = _.nin(blockedUserIds);
36	287	console.log('裸翅 [getPostList] 娣诿姑濡牠馊縡儿枊困护鋹'欢铸玩openid铸笱紆琚ㄣ院鏄囙界馊鋹
37	288	锋噫:', blockedUserIds.length);
38	289	}
39	290	
40	291	console.log('裸翅 [getPostList] 鏈€縡堀岔闾爰潄潄?', JSON.stringify(matchConditions, null, 2
41	292	));
42	293	
43	294	// 濡俗漶鏈爰岔闾爰潄潄讹紆萼旂馊match
44	295	if (Object.keys(matchConditions).length > 0) {
45	296	query = query.match(matchConditions);
46	297	}
47	298	
48	299	// 鏄ㄣ枊鐳ㄣ怛鋹ㄣ馊閫�氥満娣峰峰怛閫�昏總
49	300	// 縡ㄣ浴縡縡瘥病鏈爰岔闾爰潄潄舵禄铸塥縡縡洪) 閫ㄣ級铸岫堙鋹ㄣ馊閫�氥満娣峰峰怛

01	298	const hasFilter = isPoem !== undefined isOriginal !== undefined isDiscussion !== undef
02	↳	ined tag author includeActivity === true !!safeActivityId;
03	299	const enableRandomMix = !hasFilter; // 婁℃沿绛涙€爰潯潯舩禄錫㊀駁闌忔滿娣峰悒
04	300	
05	301	let posts = [];
06	302	let timeOrderedPosts = []; // 鎗厶㊀滿倣0 鐫庸紅潯ヤ究鎗儿欄寒編綢鍂烘禄浣跨駁
07	303	let randomPosts = []; // 鎗厶㊀滿倣0 鐫庸紅潯ヤ究鎗儿欄寒編綢鍂烘禄浣跨駁
08	304	
09	305	if (enableRandomMix) {
10	306	// 寒玲滿楞甸潰鑄氬娇壁㊀婦鏈烘販錫莖€昏總鑄?涓㊀禄闌撮『寨?+ 4涓㊀婦鏈猴級
11	307	const TIME_ORDERED_COUNT = 6; // 鑄爰禄闌撮『寨伙宛甯棧璫鏤伴嘶
12	308	const RANDOM_COUNT = 4; // 闌�滿甯棧璫鏤勹愰闌?
13	309	
14	310	// 1. 錦垺嶂鑒欵寧鐵墮梔楞哄簣鑽勹筭瀛悒紙6涓㊀級
15	311	const timeOrderedQuery = query.sort({ createTime: -1 })
16	312	.skip(skip)
17	313	.limit(TIME_ORDERED_COUNT);
18	314	
19	315	const timeOrderedRes = await timeOrderedQuery.end();
20	316	timeOrderedPosts = timeOrderedRes.list []; // 浣跨駁澶攰棧涇澹版霖鑽勹彊闌?
21	317	
22	318	// 2. 鑾峰璫闌�滿甯棧璫鑄?涓㊀級
23	319	randomPosts = []; // 閿嶈疆闌�滿甯棧璫鏤扮牦
24	320	const timeOrderedPostIds = timeOrderedPosts.map(p => p._id).filter(Boolean);
25	321	
26	322	// 錦充娇婁℃沿鐵墮梔楞哄簣鑽勹筭瀛悒紅潯燄敵璇疊嶂鑒桷婦鏈哄叺瀛?
27	323	try {
28	324	// 鐫�勹緩闌�滿甯棧璫鏤ヨ㊀鐵'欢鑄塚怩闌㰗凡鑾峰璫鑽勹禄闌撮『寨杆筭瀛悒級
29	325	const randomPostsMatchConditions = {
30	326	isHidden: _.neq(true),
31	327	isActivityPost: _.neq(true)
32	328	};
33	329	
34	330	// 澶熬埜錄€鏈爰瓮闌爰潯潯訛紙銅界玊杩欏寨旋㊀婁℃沿绛涙€爰潯潯訛紅浣嘑负浜嘑綠闌㊀繕鐫�㊀
35	↳	錄訛級
36	331	// 濡俗灘鏈爰禄闌撮『寨伙宛甯棧璫鑄岬怩闌㰗畛潯?
37	332	if (timeOrderedPostIds.length > 0) {
38	333	randomPostsMatchConditions._id = _.nin(timeOrderedPostIds);
39	334	}
40	335	
41	336	// 鎗掳橫琚㊀皖鏤界宛壁儿垺
42	337	if (blockedUserIds.length > 0) {
43	338	randomPostsMatchConditions._openid = _.nin(blockedUserIds);
44	339	}
45	340	
46	341	// 錦垺裊聰\$㊀錫塚潯潯剝宛甯棧璫鏤绘愰
47	342	const countRes = await db.collection('posts').aggregate()
48	343	.match(randomPostsMatchConditions)
49	344	.count('total')

涉及文件: functions/getPostList/index.js

回车键Poementer软件

源码范围: functions/getPostList/index.js:346 -> functions/getPostList/index.js:392

01	346	const totalPosts = (countRes.list && countRes.list[0] && countRes.list[0].total) 0;
02	347	
03	348	if (totalPosts > 0) {
04	349	// 鈇硅繙鑽劬殍鏈虹繇鐙ワ細澶氣閬闈�満鑽ヨ浣ヨ鏃�疂殍鏈烘€?
05	350	let candidatePosts = [];
06	351	const targetCandidates = Math.min(totalPosts, RANDOM_COUNT * 5); // 鑾峰彌鏃�村鏈鍊樅变
07	352	互澶烺姑闈�満鑽?
08	353	
09	354	// 濡俗澹鎢绘暱杈冨奸鏄出沔繇ヨ嶃鑼欁滃?
10	354	if (totalPosts <= targetCandidates) {
11	355	const allQuery = db.collection('posts').aggregate()
12	356	.match(randomPostsMatchConditions)
13	357	.limit(totalPosts);
14	358	const allRes = await allQuery.end();
15	359	candidatePosts = allRes.list [];
16	360	} else {
17	361	// 澶氣閬闈�満鑽ヨ鏄岃瘡娆'拱涓涘恠浠澶嶻鑾峰彌
18	362	const queriesPerBatch = 3; // 録嘆搵鑽ヨ
19	363	const perBatchCount = Math.ceil(targetCandidates / queriesPerBatch);
20	364	
21	365	for (let i = 0; i < queriesPerBatch && candidatePosts.length < targetCandidates; i++)
22	366	{
23	367	// 姣�閬闈�満闂爀嫻ヨ涓孫kip浣嶃
24	367	const maxSkip = Math.max(0, totalPosts - perBatchCount);
25	368	const randomSkip = Math.floor(Math.random() * (maxSkip + 1));
26	369	
27	370	const randomQuery = db.collection('posts').aggregate()
28	371	.match(randomPostsMatchConditions)
29	372	.sort({ createTime: -1 }) // 鏂界玢鑽爀禄闈�帹摹氬浘浣kip鏃殍鏈虹殑
30	373	.skip(randomSkip)
31	374	.limit(perBatchCount);
32	375	
33	376	const randomRes = await randomQuery.end();
34	377	const batchPosts = randomRes.list [];
35	378	
36	379	// 娑诲姑録板€樼€夂玢琛€紕蹇蹂嶃闈?
37	380	const existingIds = new Set(candidatePosts.map(p => p._id));
38	381	const newPosts = batchPosts.filter(p => !existingIds.has(p._id));
39	382	candidatePosts = candidatePosts.concat(newPosts);
40	383	}
41	384	}
42	385	
43	386	// 濡俗澹鍊樼€夂笱瀛�繑涓堣鏄�拱録€鏄笱瀛��閬闈�満闂爀嫻skip浣�鑾將嶃鑼?
44	387	if (candidatePosts.length < RANDOM_COUNT && totalPosts > candidatePosts.length) {
45	388	const remainingNeeded = RANDOM_COUNT - candidatePosts.length;
46	389	const existingIds = new Set(candidatePosts.map(p => p._id));
47	390	
48	391	// 鍑堟敵璜囡殍娆殑殍鏈烘健璇?
49	392	for (let attempt = 0; attempt < 5 && candidatePosts.length < targetCandidates; attem

涉及文件: functions/getPostList/index.js

回车键Poementer软件

源码范围: functions/getPostList/index.js:393 -> functions/getPostList/index.js:441

01	393	const maxSkip = Math.max(0, totalPosts - remainingNeeded * 2);
02	394	const randomSkip = Math.floor(Math.random() * (maxSkip + 1));
03	395	
04	396	const randomQuery = db.collection('posts').aggregate()
05	397	.match(randomPostsMatchConditions)
06	398	.sort({ createTime: -1 })
07	399	.skip(randomSkip)
08	400	.limit(remainingNeeded * 2);
09	401	
10	402	const randomRes = await randomQuery.end();
11	403	const batchPosts = randomRes.list [];
12	404	const newPosts = batchPosts.filter(p => !existingIds.has(p._id));
13	405	candidatePosts = candidatePosts.concat(newPosts);
14	406	newPosts.forEach(p => existingIds.add(p._id));
15	407	}
16	408	}
17	409	
18	410	// 湔庾€櫳€父笱瀛愨脰褰悔簪闾忓満録撲貢騫墮€爻嫫鏊堦娇璩?Fisher-Yates shuffle 纁格磅镑
19	411	?
20	411	const shuffled = candidatePosts.slice();
21	412	for (let i = shuffled.length - 1; i > 0; i--) {
22	413	const j = Math.floor(Math.random() * (i + 1));
23	414	const temp = shuffled[i];
24	415	shuffled[i] = shuffled[j];
25	416	shuffled[j] = temp;
26	417	}
27	418	randomPosts = shuffled.slice(0, Math.min(RANDOM_COUNT, shuffled.length));
28	419	}
29	420	} catch (randomError) {
30	421	console.error('鉁?[getPostList] 璩峰璩闾忓満甯栨璩澶犛触:', randomError);
31	422	}
32	423	
33	424	// 3. 娉峰惜甯栨璩璩氮隅姝 i 殢鏈烱販錫塚禄闾撮『蹇杆拒闾忓満甯栨璩
34	425	if (randomPosts.length > 0) {
35	426	// 録湲纁録€鏈€父笱瀛愨脰錫塢苟録脩 "
36	427	const allPosts = timeOrderedPosts.concat(randomPosts);
37	428	
38	429	// 浣跨敳 Fisher-Yates shuffle 纁格磅镑燔闾忓満録撲貢
39	430	for (let i = allPosts.length - 1; i > 0; i--) {
40	431	const j = Math.floor(Math.random() * (i + 1));
41	432	const temp = allPosts[i];
42	433	allPosts[i] = allPosts[j];
43	434	allPosts[j] = temp;
44	435	}
45	436	
46	437	posts = allPosts;
47	438	} else {
48	439	posts = timeOrderedPosts.slice();
49	440	}

01	442	// 滿卞推疊閤細鑒綉鏤映綠闌撮「摹伙苑甯棧璣
02	443	const timeOrderedQuery = query.sort({ createTime: -1 })
03	444	.skip(skip)
04	445	.limit(limit);
05	446	
06	447	const timeOrderedRes = await timeOrderedQuery.end();
07	448	posts = timeOrderedRes.list [];
08	449	}
09	450	
10	451	// 纒縹杓旻誤鑽勸笥瀛愔噉閱忒笏璣皆縹limit
11	452	posts = posts.slice(0, limit);
12	453	
13	454	// 錄归嘶鑊ヨ錄硅褱鐘舵€?
14	455	let voterMap = new Set();
15	456	if (posts.length > 0) {
16	457	try {
17	458	const postIds = posts.map(post => post._id);
18	459	const voteRes = await db.collection('votes_log')
19	460	.where({
20	461	_openid: openid,
21	462	type: 'post',
22	463	postId: _.in(postIds)
23	464	})
24	465	.field({ postId: true })
25	466	.get();
26	467	voterMap = new Set(voteRes.data.map(item => item.postId));
27	468	} catch (voteError) {
28	469	console.error('鉞?[getPostList] 錄归嘶鑊ヨ錄硅褱璉板絳澶弊触:', voteError);
29	470	}
30	471	}
31	472	
32	473	// 澶勦愆甯棧璣鑲版嶷铸岢苟鏤槽囧杓困护琨皖繡界駁鎡凤紙縑明噸淇漣櫓铸?
33	474	let processedPosts = posts.map(post => {
34	475	const authorName = post.authorName post.authorNameSnapshot '錕垮悒鑿儿垲';
35	476	const authorAvatar = post.authorAvatar post.authorAvatarSnapshot '';
36	477	const authorSignature = post.authorSignature ''; // 绛悒悒URL铸塢鷹錫莖笥藏愬影鑊戒负纒
37	478	猴級
38	479	const commentCount = post.commentCount 0;
39	480	const isVoted = voterMap.has(post._id);
40	481	
41	482	// 緇勸癢瀛格鏤錚滄簪铸氩‘淇漬垺琛乂綉鏤炧噉瀨賤墀涓爰
42	483	const seriesBlocksRaw = Array.isArray(post.seriesBlocks) ? post.seriesBlocks : [];
43	484	const seriesBlocks = seriesBlocksRaw.slice(0, 3).map((b, idx) => ({
44	485	id: b.id `series-\${idx}`,
45	486	subtitle: b.subtitle b.subTitle '',
46	487	content: b.content '',
47	488	highlightSentence: b.highlightSentence (b.content ? String(b.content).split(/\r?\n/).
48	489	find(l => l && l.trim()) '' : ''),
49	490	highlightLines: Array.isArray(b.highlightLines) ? b.highlightLines : []

涉及文件: functions/getPostList/index.js

回车键Poementer软件

源码范围: functions/getPostList/index.js:490 -> functions/getPostList/index.js:535

01	490	const seriesBlockCount = post.seriesBlockCount seriesBlocksRaw.length seriesBlocks.l
02	491	ength;
03	492	const isSeries = post.isSeries seriesBlocks.length > 0;
04	493	
05	494	return {
06	495	...post,
07	496	isSeries,
08	497	seriesBlocks,
09	498	seriesBlockCount,
10	499	authorName,
11	500	authorAvatar,
12	501	authorSignature,
13	502	commentCount,
14	503	isVoted,
15	504	tags: Array.isArray(post.tags) ? post.tags : []
16	505	};
17	506	});
18	507	
19	508	// 鑒明噸淇澁櫓銑氫埤纒㊦哇娆㊦繃婊㊦瀾他駁鑿儿垠鑽勸笥瀛脩紙鋸呶罇鋸垮悒棧璵鑽刼ealAuthor0
20	509	penid鑄?
21	510	if (blockedUserIds.length > 0) {
22	511	processedPosts = processedPosts.filter(post => {
23	512	// 妨€鑛ㄣ櫓閭氫笥瀛悽宛 _openid
24	513	if (blockedUserIds.includes(post._openid)) {
25	514	return false;
26	515	}
27	516	// 妨€鑛ㄣ鷹錫整笥瀛悽宛 realAuthorOpenid
28	517	if (post.realAuthorOpenid && blockedUserIds.includes(post.realAuthorOpenid)) {
29	518	return false;
30	519	}
31	520	return true;
32	521	});
33	522	}
34	523	
35	524	// --- 浼樺窳鍍劇塢URL杞㊦嶢閭昏總 ---
36	525	const fileIDs = new Set(); // 浣跨駁Set閭垮厓閱整㊦fileID
37	526	
38	527	processedPosts.forEach(post => {
39	528	// 淇濈痼 imageUrls鎔莖riginalImageUrls 涓€瀹�氫笥鑛扮牦
40	529	if (!Array.isArray(post.imageUrls)) post.imageUrls = post.imageUrls ? [post.imageUrls] : [
41	530	];
42	531	if (!Array.isArray(post.originalImageUrls)) post.originalImageUrls = post.originalImageUr
43	532	l s ? [post.originalImageUrls] : [];
44	533	
45	534	// 鎔墮汎鎔€鏈變復漢瑕伙漿鎚㊦宛fileID
46	535	const urlsToCheck = [
47	536	...post.imageUrls,
48	537	...post.originalImageUrls,
49	538	post.imageUrl,

涉及文件: functions/getPostList/index.js

回车键Poementer软件

源码范围: functions/getPostList/index.js:536 -> functions/getPostList/index.js:584

01	536	post.authorAvatar,
02	537	post.authorSignature, // 娣诲姑绛惧恹URL
03	538	post.poemBgImage
04	539	].filter(url => url && url.startsWith('cloud://'));
05	540	
06	541	urlsToCheck.forEach(url => fileIDs.add(url));
07	542	});
08	543	
09	544	if (fileIDs.size > 0) {
10	545	try {
11	546	const fileListResult = await cloud.getTempFileURL({ fileList: Array.from(fileIDs) });
12	547	const urlMap = new Map();
13	548	
14	549	fileListResult.fileList.forEach(item => {
15	550	if (item.status === 0) {
16	551	urlMap.set(item.fileID, item.tempFileURL);
17	552	}
18	553	});
19	554	
20	555	// 錄归嘶杞嶲錄€鏈父筈瀛愢殒鏹剧墁URL
21	556	processedPosts.forEach(post => {
22	557	const convertUrl = (url) => urlMap.get(url) url;
23	558	
24	559	if (post.imageUrl) post.imageUrl = convertUrl(post.imageUrl);
25	560	if (post.originalImageUrl) post.originalImageUrl = convertUrl(post.originalImageUrl);
26	561	if (post.authorAvatar) post.authorAvatar = convertUrl(post.authorAvatar);
27	562	if (post.authorSignature) post.authorSignature = convertUrl(post.authorSignature); //
28	563	杞嶲绛惧恹URL
29	563	if (post.poemBgImage) post.poemBgImage = convertUrl(post.poemBgImage);
30	564	
31	565	if (Array.isArray(post.imageUrls)) {
32	566	post.imageUrls = post.imageUrls.map(convertUrl);
33	567	}
34	568	if (Array.isArray(post.originalImageUrls)) {
35	569	post.originalImageUrls = post.originalImageUrls.map(convertUrl);
36	570	}
37	571	});
38	572	} catch (fileError) {
39	573	console.error('鏹?[getPostList] 鏹剧墁URL杞嶲澶嶿辨馐:', fileError);
40	574	}
41	575	}
42	576	
43	577	console.log('鈱?[getPostList] ===== 浜戱嚟鐗鐗鐗琛�畨鐗?=====');
44	578	console.log('鈱?[getPostList] 杩旂湅甯�棧鐗鐗鐗浣�馐:', processedPosts.length);
45	579	if (processedPosts.length > 0) {
46	580	console.log('鈱?[getPostList] 鐗?涓?筈瀛�闂?', processedPosts.slice(0, 3).map(p => ({
47	581	id: p._id,
48	582	createTime: p.createTime
49	583	}}));

涉及文件: functions/getPostList/index.js -> functions/getPostDetail/index.js (共 2 个文件)

回车键Poementer软件

源码范围: functions/getPostList/index.js:585 -> functions/getPostDetail/index.js:26

01	585	
02	586	return {
03	587	success: true,
04	588	posts: processedPosts
05	589	};
06	590	
07	591	} catch (e) {
08	592	console.error('鉅?[getPostList] 浜戠殑鐢靛版鍒嗘瀽け璫?', e);
09	593	console.error('鉅?[getPostList] 閿欏洑璇峵儻:', {
10	594	message: e.message,
11	595	stack: e.stack,
12	596	name: e.name
13	597	});
14	598	return {
15	599	success: false,
16	600	error: {
17	601	message: e.message,
18	602	stack: e.stack
19	603	}
20	604	};
21	605	}
22	606	};
23	607	
24	FILE FILE 14 functions/getPostDetail/index.js 帖子详情云函数	
25	1	// 云函数入口文件
26	2	const cloud = require('wx-server-sdk');
27	3	
28	4	cloud.init({
29	5	env: cloud.DYNAMIC_CURRENT_ENV
30	6	});
31	7	
32	8	const db = cloud.database();
33	9	
34	10	// 云函数入口函数
35	11	exports.main = async (event, context) => {
36	12	const wxContext = cloud.getWXContext();
37	13	const wxCtxOpenid = wxContext.OPENID;
38	14	const eventOpenid = event.openid;
39	15	const openid = eventOpenid wxCtxOpenid;
40	16	const { postId } = event;
41	17	
42	18	if (!openid) {
43	19	return {
44	20	success: false,
45	21	message: '无法获取用户 openid, 请重新登录',
46	22	code: 'NO_OPENID'
47	23	};
48	24	}
49	25	console.log('🔗 [getPostDetail] openid来源:', {

涉及文件: functions/getPostDetail/index.js

回车键Poementer软件

源码范围: functions/getPostDetail/index.js:27 -> functions/getPostDetail/index.js:76

01	27	wxCtxOpenid: wxCtxOpenid ? '提供' : '未提供',
02	28	chosenOpenidSource: eventOpenid ? 'event.openid' : 'wxContext.OPENID',
03	29	chosenOpenidExists: !!openid
04	30	});
05	31	
06	32	if (!postId) {
07	33	return {
08	34	success: false,
09	35	message: 'Post ID is required.'
10	36	};
11	37	}
12	38	
13	39	try {
14	40	// 1. 根据 postId 获取帖子详情
15	41	const postRes = await db.collection('posts').doc(postId).get();
16	42	const post = postRes.data;
17	43	// 如果帖子被隐藏且当前用户不是作者, 禁止访问
18	44	if (post && post.isHidden === true && post._openid !== openid) {
19	45	return {
20	46	success: false,
21	47	code: 'POST_HIDDEN',
22	48	message: '该内容已隐藏'
23	49	};
24	50	}
25	51	
26	52	if (!post) {
27	53	return {
28	54	success: false,
29	55	message: 'Post not found.'
30	56	};
31	57	}
32	58	
33	59	console.log('获取到的帖子数据:', {
34	60	_id: post._id,
35	61	title: post.title,
36	62	commentCount: post.commentCount,
37	63	votes: post.votes
38	64	});
39	65	
40	66	// 2. 根据帖子的 _openid 获取作者信息
41	67	const userRes = await db.collection('users').where({
42	68	_openid: post._openid
43	69	}).get();
44	70	
45	71	// If author is not found, provide a default object for robustness
46	72	const author = userRes.data[0] {
47	73	nickName: '匿名用户',
48	74	avatarUrl: ''
49	75	};

涉及文件: functions/getPostDetail/index.js

回车键Poementer软件

源码范围: functions/getPostDetail/index.js:77 -> functions/getPostDetail/index.js:124

01	77	// 3. 获取当前用户的点赞记录
02	78	const voteRes = await db.collection('votes_log').where({
03	79	_openid: openid,
04	80	postId: postId,
05	81	type: 'post'
06	82	}).get();
07	83	
08	84	// 4. 组合最终结果
09	85	const resultPost = {
10	86	...post,
11	87	authorName: post.authorName post.authorNameSnapshot author.nickName '匿名用户',
12	88	authorAvatar: post.authorAvatar post.authorAvatarSnapshot author.avatarUrl '',
13	89	authorSignature: post.authorSignature '', // 签名URL (优先使用帖子中存储的值)
14	90	isAuthor: post._openid === openid,
15	91	isVoted: voteRes.data.length > 0,
16	92	tags: post.tags [] // 确保标签字段存在
17	93	};
18	94	// 保证 imageUrls、originalImageUrls 一定为数组
19	95	if (!Array.isArray(resultPost.imageUrls)) resultPost.imageUrls = resultPost.imageUrls ? [res
20	96	ultPost.imageUrls] : [];
21	96	if (!Array.isArray(resultPost.originalImageUrls)) resultPost.originalImageUrls = resultPost.
22	97	originalImageUrls ? [resultPost.originalImageUrls] : [];
23	97	
24	98	// --- Efficiently convert FileIDs to temp URLs ---
25	99	const fileIDs = [];
26	100	if (resultPost.imageUrl && resultPost.imageUrl.startsWith('cloud://')) {
27	101	fileIDs.push(resultPost.imageUrl);
28	102	}
29	103	if (resultPost.imageUrls && Array.isArray(resultPost.imageUrls)) {
30	104	resultPost.imageUrls.forEach(url => {
31	105	if (url && url.startsWith('cloud://')) {
32	106	fileIDs.push(url);
33	107	}
34	108	});
35	109	}
36	110	if (resultPost.originalImageUrl && resultPost.originalImageUrl.startsWith('cloud://')) {
37	111	fileIDs.push(resultPost.originalImageUrl);
38	112	}
39	113	if (resultPost.originalImageUrls && Array.isArray(resultPost.originalImageUrls)) {
40	114	resultPost.originalImageUrls.forEach(url => {
41	115	if (url && url.startsWith('cloud://')) {
42	116	fileIDs.push(url);
43	117	}
44	118	});
45	119	}
46	120	if (resultPost.authorAvatar && resultPost.authorAvatar.startsWith('cloud://')) {
47	121	fileIDs.push(resultPost.authorAvatar);
48	122	}
49	123	if (resultPost.authorSignature && resultPost.authorSignature.startsWith('cloud://')) {

涉及文件: functions/getPostDetail/index.js

回车键Poementer软件

源码范围: functions/getPostDetail/index.js:125 -> functions/getPostDetail/index.js:174

01	125	}
02	126	
03	127	if (fileIDs.length > 0) {
04	128	const fileListResult = await cloud.getTempFileURL({ fileList: fileIDs });
05	129	const urlMap = new Map();
06	130	fileListResult.fileList.forEach(item => {
07	131	if (item.status === 0) {
08	132	urlMap.set(item.fileID, item.tempFileURL);
09	133	}
10	134	});
11	135	
12	136	if (resultPost.imageUrl && urlMap.has(resultPost.imageUrl)) {
13	137	resultPost.imageUrl = urlMap.get(resultPost.imageUrl);
14	138	}
15	139	if (resultPost.imageUrls && Array.isArray(resultPost.imageUrls)) {
16	140	resultPost.imageUrls = resultPost.imageUrls.map(url =>
17	141	url && urlMap.has(url) ? urlMap.get(url) : url
18	142	);
19	143	}
20	144	if (resultPost.originalImageUrl && urlMap.has(resultPost.originalImageUrl)) {
21	145	resultPost.originalImageUrl = urlMap.get(resultPost.originalImageUrl);
22	146	}
23	147	if (resultPost.originalImageUrls && Array.isArray(resultPost.originalImageUrls)) {
24	148	resultPost.originalImageUrls = resultPost.originalImageUrls.map(url =>
25	149	url && urlMap.has(url) ? urlMap.get(url) : url
26	150	);
27	151	}
28	152	if (resultPost.authorAvatar && urlMap.has(resultPost.authorAvatar)) {
29	153	resultPost.authorAvatar = urlMap.get(resultPost.authorAvatar);
30	154	}
31	155	if (resultPost.authorSignature && urlMap.has(resultPost.authorSignature)) {
32	156	resultPost.authorSignature = urlMap.get(resultPost.authorSignature);
33	157	}
34	158	}
35	159	
36	160	// 如果帖子没有commentCount字段, 则实时计算评论数量
37	161	let commentCount = resultPost.commentCount;
38	162	console.log('从数据库获取的commentCount:', commentCount, '类型:', typeof commentCount);
39	163	
40	164	if (commentCount === undefined commentCount === null) {
41	165	console.log('需要实时计算评论数量');
42	166	const commentsRes = await db.collection('comments').where({
43	167	postId: postId
44	168	}).count();
45	169	commentCount = commentsRes.total;
46	170	console.log('实时计算的评论数量:', commentCount);
47	171	
48	172	// 同时更新帖子数据库中的commentCount字段, 避免下次再计算
49	173	try {

01	175	data: {
02	176	commentCount: commentCount
03	177	}
04	178	});
05	179	console.log('已更新数据库中的commentCount字段');
06	180	} catch (updateError) {
07	181	console.error('更新commentCount字段失败:', updateError);
08	182	}
09	183	} else {
10	184	console.log('使用数据库中的commentCount:', commentCount);
11	185	}
12	186	
13	187	console.log('最终返回的评论数量:', commentCount);
14	188	
15	189	return {
16	190	post: resultPost,
17	191	commentCount: commentCount, // 添加评论数量
18	192	success: true
19	193	};
20	194	
21	195	} catch (e) {
22	196	console.error(e);
23	197	return {
24	198	success: false,
25	199	error: e
26	200	};
27	201	}
28	202	};
29	FILE FILE 15 functions/vote/index.js 点赞云函数	
30	1	const cloud = require('wx-server-sdk')
31	2	cloud.init({ env: cloud.DYNAMIC_CURRENT_ENV })
32	3	
33	4	const db = cloud.database()
34	5	const _ = db.command
35	6	
36	7	// buckets for growth stats on user profile
37	8	// 统一阈值: 与前端显示一致 (1-3/4-7/8-15/16+)
38	9	const BUCKETS = [
39	10	{ key: 'seed', min: 1, max: 3 },
40	11	{ key: 'leaf', min: 4, max: 7 },
41	12	{ key: 'flower', min: 8, max: 15 },
42	13	{ key: 'peach', min: 16, max: Infinity },
43	14	]
44	15	const bucketOf = (v) => {
45	16	v = typeof v === 'number' ? v : 0
46	17	if (v < 1) return null
47	18	for (const b of BUCKETS) {
48	19	if (v >= b.min && v <= b.max) return b.key
49	20	}

涉及文件: functions/vote/index.js

回车键Poementer软件

源码范围: functions/vote/index.js:22 -> functions/vote/index.js:71

01	22	}
02	23	
03	24	async function updateAuthorGrowthCounts(authorOpenid, oldVotes, newVotes) {
04	25	const from = bucketOf(oldVotes)
05	26	const to = bucketOf(newVotes)
06	27	if (from === to) return
07	28	const data = { growthUpdatedAt: db.serverDate() }
08	29	if (from) data[`growthCounts.\${from}`] = _.inc(-1)
09	30	if (to) data[`growthCounts.\${to}`] = _.inc(1)
10	31	const upd = await db.collection('users').where({ _openid: authorOpenid }).update({ data })
11	32	if (!upd.stats upd.stats.updated === 0) {
12	33	try {
13	34	await db.collection('users').add({
14	35	data: {
15	36	_openid: authorOpenid,
16	37	growthCounts: {
17	38	seed: to === 'seed' ? 1 : 0,
18	39	leaf: to === 'leaf' ? 1 : 0,
19	40	flower: to === 'flower' ? 1 : 0,
20	41	peach: to === 'peach' ? 1 : 0,
21	42	},
22	43	growthUpdatedAt: db.serverDate(),
23	44	createTime: new Date(),
24	45	updateTime: new Date(),
25	46	},
26	47	})
27	48	} catch (e) {
28	49	// ignore race
29	50	}
30	51	}
31	52	}
32	53	
33	54	// 云函数入口函数
34	55	exports.main = async (event, context) => {
35	56	
36	57	try {
37	58	const { postId } = event
38	59	const wxContext = cloud.getWXContext()
39	60	const wxCtxOpenid = wxContext.OPENID
40	61	const eventOpenid = event.openid
41	62	const openid = eventOpenid wxCtxOpenid
42	63	
43	64	// 读取帖子, 拿作者与当前票数 (用于分段迁移)
44	65	const postSnapBefore = await db.collection('posts').doc(postId).get()
45	66	const postBefore = postSnapBefore.data
46	67	if (!postBefore) {
47	68	return { success: false, message: 'POST_NOT_FOUND' }
48	69	}
49	70	const authorOpenid = postBefore._openid

涉及文件: functions/vote/index.js

回车键Poementer软件

源码范围: functions/vote/index.js:72 -> functions/vote/index.js:121

01	72	
02	73	if (!openid) {
03	74	console.error('✖ [vote] 无法获取用户 openid');
04	75	return {
05	76	success: false,
06	77	message: '无法获取用户 openid, 请重新登录',
07	78	code: 'NO_OPENID'
08	79	}
09	80	}
10	81	
11	82	// 1. 查找 votes_log 表, 精确查找 type 为 'post' 的记录
12	83	const log = await db.collection('votes_log').where({
13	84	_openid: openid,
14	85	postId: postId,
15	86	type: 'post'
16	87	}).get()
17	88	
18	89	
19	90	let updatedPost;
20	91	let isLiked = false;
21	92	
22	93	if (log.data.length > 0) {
23	94	// 2. 如果找到了记录, 说明是"取消点赞"
24	95	await db.collection('votes_log').doc(log.data[0]._id).remove()
25	96	await db.collection('posts').doc(postId).update({ data: { votes: _.inc(-1) } })
26	97	const postSnapAfter = await db.collection('posts').doc(postId).get()
27	98	const newVotes = (postSnapAfter.data && postSnapAfter.data.votes) (oldVotes - 1)
28	99	await updateAuthorGrowthCounts(authorOpenid, oldVotes, newVotes)
29	100	isLiked = false
30	101	} else {
31	102	// 3. 如果没找到记录, 说明是"点赞"
32	103	await db.collection('votes_log').add({
33	104	data: {
34	105	_openid: openid,
35	106	postId: postId,
36	107	type: 'post',
37	108	createTime: new Date()
38	109	}
39	110	})
40	111	await db.collection('posts').doc(postId).update({ data: { votes: _.inc(1) } })
41	112	const postSnapAfter = await db.collection('posts').doc(postId).get()
42	113	const newVotes = (postSnapAfter.data && postSnapAfter.data.votes) (oldVotes + 1)
43	114	await updateAuthorGrowthCounts(authorOpenid, oldVotes, newVotes)
44	115	isLiked = true
45	116	
46	117	// === 新增: 创建点赞消息通知 ===
47	118	try {
48	119	// 获取帖子信息
49	120	const postResult = await db.collection('posts').doc(postId).get()

涉及文件: functions/vote/index.js

回车键Poementer软件

源码范围: functions/vote/index.js:122 -> functions/vote/index.js:171

01	122	
02	123	// 获取点赞者信息
03	124	const userResult = await db.collection('users').where({
04	125	_openid: openid
05	126	}).limit(1).get()
06	127	const user = userResult.data[0]
07	128	
08	129	// 如果给自己点赞, 不发送通知
09	130	if (post._openid !== openid) {
10	131	// 根据帖子实际字段确定内容类型
11	132	let contentType = 'post';
12	133	let contentTypeText = '帖子';
13	134	
14	135	if (post.isDiscussion) {
15	136	contentType = 'discussion';
16	137	contentTypeText = '讨论';
17	138	} else if (post.isPoem) {
18	139	if (post.isOriginal) {
19	140	contentType = 'original';
20	141	contentTypeText = '原创诗歌';
21	142	} else {
22	143	contentType = 'non-original';
23	144	contentTypeText = '诗歌';
24	145	}
25	146	}
26	147	
27	148	await db.collection('messages').add({
28	149	data: {
29	150	fromUserId: openid,
30	151	fromUserName: user ? user.nickName : '微信用户',
31	152	fromUserAvatar: user ? user.avatarUrl : '',
32	153	toUserId: post._openid,
33	154	type: 'like',
34	155	postId: postId,
35	156	postTitle: post.title '无标题',
36	157	contentType: contentType,
37	158	content: `\${user ? user.nickName : '微信用户'} 点赞了你的\${contentTypeText}`,
38	159	isRead: false,
39	160	createTime: new Date()
40	161	}
41	162	})
42	163	}
43	164	} catch (msgError) {
44	165	console.error('创建点赞消息失败:', msgError)
45	166	// 不影响主流程, 只是记录错误
46	167	}
47	168	}
48	169	
49	170	// 4. 无论点赞还是取消, 都重新获取文章的最新数据

提交页：56 / 60

完整序列页：281 / 285

涉及文件：functions/vote/index.js -> functions/addComment/index.js（共 2 个文件）

回车键Poementer软件

源码范围：functions/vote/index.js:172 -> functions/addComment/index.js:27

01	172	
02	173	const finalVotes = updatedPost.data.votes 0;
03	174	
04	175	const result = {
05	176	success: true,
06	177	votes: finalVotes, // 返回最新的点赞数
07	178	isLiked: isLiked
08	179	};
09	180	
10	181	return result;
11	182	
12	183	} catch (e) {
13	184	console.error('云函数执行失败:', e);
14	185	return {
15	186	success: false,
16	187	error: {
17	188	message: e.message,
18	189	stack: e.stack
19	190	}
20	191	}
21	192	}
22	193	}
23	FILE FILE 16 functions/addComment/index.js 评论云函数	
24	1	// 云函数 addComment 的入口文件（最终版）
25	2	const cloud = require('wx-server-sdk');
26	3	
27	4	cloud.init({
28	5	env: cloud.DYNAMIC_CURRENT_ENV
29	6	});
30	7	
31	8	const db = cloud.database();
32	9	const _ = db.command; // 引入数据库操作符
33	10	
34	11	// 云函数入口函数
35	12	exports.main = async (event, context) => {
36	13	const wxContext = cloud.getWXContext();
37	14	const openid = wxContext.OPENID event.openid;
38	15	
39	16	if (!openid) {
40	17	return {
41	18	success: false,
42	19	message: '无法获取用户 openid, 请重新登录',
43	20	code: 'NO_OPENID'
44	21	};
45	22	}
46	23	
47	24	// [新增] 接收前端传来的 replyToAuthorName
48	25	const {
49	26	postId,

涉及文件: functions/addComment/index.js

回车键Poementer软件

源码范围: functions/addComment/index.js:28 -> functions/addComment/index.js:76

01	28	parentId,
02	29	replyToAuthorName,
03	30	imageUrls = [],
04	31	originalImageUrls = [],
05	32	isAnonymous = false
06	33	} = event;
07	34	
08	35	const trimmedContent = (content '').trim();
09	36	const sanitizedImageUrls = Array.isArray(imageUrls) ? imageUrls.filter(url => !!url) : [];
10	37	let sanitizedOriginalImageUrls = Array.isArray(originalImageUrls) ? originalImageUrls.filter(u
11	38	rl => !!url) : [];
12	39	
13	40	if (sanitizedOriginalImageUrls.length === 0 && sanitizedImageUrls.length > 0) {
14	41	sanitizedOriginalImageUrls = sanitizedImageUrls.slice();
15	42	}
16	43	
17	44	const hasContent = trimmedContent.length > 0;
18	45	const hasImages = sanitizedImageUrls.length > 0;
19	46	
20	47	// Basic validation
21	48	if (!postId) {
22	49	return { success: false, message: 'Post ID is required.' };
23	50	}
24	51	if (!hasContent && !hasImages) {
25	52	return { success: false, message: '评论内容或图片至少需要一项。' };
26	53	}
27	54	
28	55	try {
29	56	// 获取用户信息
30	57	const userResult = await db.collection('users').where({
31	58	_openid: openid
32	59	).limit(1).get();
33	60	const user = userResult.data[0];
34	61	
35	62	// 准备要存入数据库的数据
36	63	const commentData = {
37	64	_openid: openid,
38	65	postId: postId,
39	66	content: trimmedContent,
40	67	likes: 0, // [建议] 初始化点赞数为0
41	68	createTime: new Date(),
42	69	imageUrls: sanitizedImageUrls,
43	70	originalImageUrls: sanitizedOriginalImageUrls,
44	71	hasImages: hasImages,
45	72	// 匿名评论相关字段
46	73	isAnonymous: isAnonymous,
47	74	authorName: isAnonymous ? '匿名用户' : (user ? user.nickName : '微信用户'),
48	75	authorAvatar: isAnonymous ? '/static/images/avatar.png' : (user ? user.avatarUrl : ''),
49		realAuthorOpenid: isAnonymous ? openid : null

涉及文件: functions/addComment/index.js

回车键Poementer软件

源码范围: functions/addComment/index.js:77 -> functions/addComment/index.js:126

01	77	
02	78	// 如果 parentId 存在, 说明这是一条回复
03	79	if (parentId) {
04	80	commentData.parentId = parentId;
05	81	// [新增] 如果是回复, 就把被回复人的名字也存进去
06	82	if (replyToAuthorName) {
07	83	commentData.replyToAuthorName = replyToAuthorName;
08	84	}
09	85	}
10	86	
11	87	// 将新评论/回复添加到数据库
12	88	const result = await db.collection('comments').add({
13	89	data: commentData
14	90	});
15	91	
16	92	// [新增] 评论或回复成功后, 帖子的评论数 +1
17	93	await db.collection('posts').doc(postId).update({
18	94	data: {
19	95	commentCount: _.inc(1)
20	96	}
21	97	});
22	98	
23	99	// === 新增: 创建评论消息通知 ===
24	100	try {
25	101	// 获取帖子信息
26	102	const postResult = await db.collection('posts').doc(postId).get()
27	103	const post = postResult.data
28	104	
29	105	let notifyUserId = null; // 要通知的用户ID
30	106	let shouldNotify = false; // 是否应该发送通知
31	107	
32	108	if (parentId) {
33	109	// 如果是回复评论, 需要通知被回复评论的作者
34	110	try {
35	111	const parentCommentResult = await db.collection('comments').doc(parentId).get();
36	112	const parentComment = parentCommentResult.data;
37	113	
38	114	if (parentComment) {
39	115	// 获取被回复评论的作者ID (如果是匿名评论, 使用realAuthorOpenid)
40	116	const parentCommentAuthorId = parentComment.isAnonymous
41	117	? (parentComment.realAuthorOpenid parentComment._openid)
42	118	: parentComment._openid;
43	119	
44	120	// 如果回复的不是自己的评论, 且不是匿名评论, 则发送通知
45	121	if (parentCommentAuthorId !== openid && !isAnonymous) {
46	122	notifyUserId = parentCommentAuthorId;
47	123	shouldNotify = true;
48	124	}
49	125	}

涉及文件: functions/addComment/index.js

回车键Poementer软件

源码范围: functions/addComment/index.js:127 -> functions/addComment/index.js:176

01	127	console.error('查询被回复评论失败:', commentError)
02	128	}
03	129	} else {
04	130	// 如果是直接评论帖子, 通知帖子作者 (排除给自己评论或匿名评论的情况)
05	131	if (post._openid !== openid && !isAnonymous) {
06	132	notifyUserId = post._openid;
07	133	shouldNotify = true;
08	134	}
09	135	}
10	136	
11	137	if (shouldNotify && notifyUserId) {
12	138	// 根据帖子实际字段确定内容类型
13	139	let contentType = 'post';
14	140	let contentTypeText = '帖子';
15	141	
16	142	if (post.isDiscussion) {
17	143	contentType = 'discussion';
18	144	contentTypeText = '讨论';
19	145	} else if (post.isPoem) {
20	146	if (post.isOriginal) {
21	147	contentType = 'original';
22	148	contentTypeText = '原创诗歌';
23	149	} else {
24	150	contentType = 'non-original';
25	151	contentTypeText = '诗歌';
26	152	}
27	153	}
28	154	
29	155	const messageContent = parentId ?
30	156	`\${user ? user.nickName : '微信用户'} 回复了你的评论` :
31	157	`\${user ? user.nickName : '微信用户'} 评论了你的\${contentTypeText}`
32	158	
33	159	await db.collection('messages').add({
34	160	data: {
35	161	fromUserId: openid,
36	162	fromUserName: user ? user.nickName : '微信用户',
37	163	fromUserAvatar: user ? user.avatarUrl : '',
38	164	toUserId: notifyUserId,
39	165	type: 'comment',
40	166	postId: postId,
41	167	postTitle: post.title '无标题',
42	168	contentType: contentType,
43	169	// 标记是否为对评论的回复, 便于前端准确展示
44	170	isReply: !!parentId,
45	171	content: messageContent,
46	172	commentId: result._id,
47	173	isRead: false,
48	174	createTime: new Date()
49	175	}

提交页: 60 / 60

完整序列页: 285 / 285

涉及文件: functions/addComment/index.js

源码范围: functions/addComment/index.js:177 -> functions/addComment/index.js:199

01

177

}

02

178

} catch (msgError) {

03

179

console.error('创建评论消息失败:', msgError)

04

180

// 不影响主流程, 只是记录错误

05

181

}

06

182

07

183

return {

08

184

success: true,

09

185

message: 'Comment added successfully.',

10

186

commentId: result._id

11

187

};

12

188

13

189

} catch (e) {

14

190

console.error('addComment error', e);

15

191

return {

16

192

success: false,

17

193

message: 'Failed to add comment.',

18

194

error: e.toString()

19

195

};

20

196

}

21

197

};

22

198

23

199

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49